Module 3

Illumination models

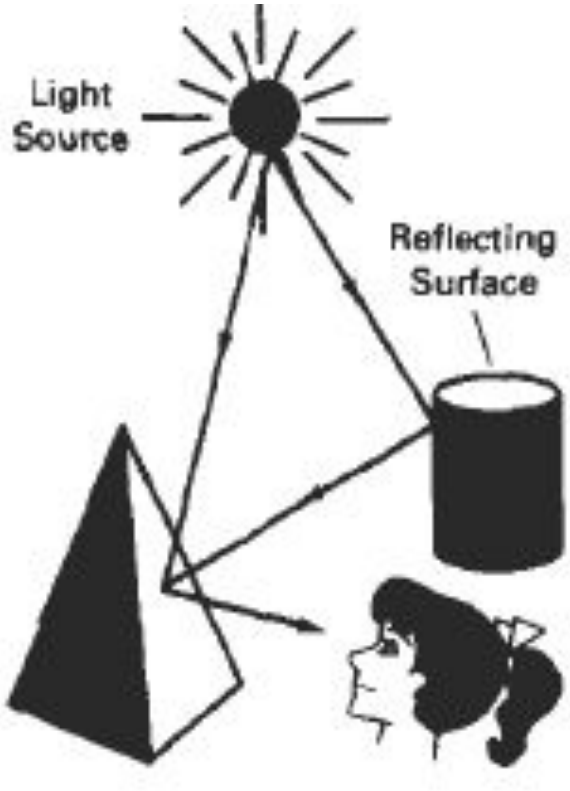
Illumination and Rendering

- ***Illumination model*** in computer graphics
  - also called a ***lighting model*** or a ***shading model***
  - used to calculate the intensity(color) of an illuminated position on the surface of an object
- ***Surface-rendering method***
  - also called a ***surface shading method***
  - determine the light intensity (pixel colors) for all projected positions in a scene using intensity calculations from an illumination model
  - Surface rendering can be performed by applying illumination model to every visible surface point, or can be accomplished by interpolating intensities across the surface from a small set of illumination-model calculations.

Light Sources

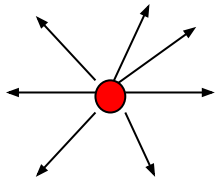
- When we view an opaque non-luminous object, we see reflected light from the surfaces of the object.
- The total reflected light is the sum of the contributions from ***light sources*** and other ***reflecting surfaces*** in the scene.
- Light source: object that emits radiant energy
- Light sources = ***light-emitting sources***. (bulb, sun)
- Reflecting surfaces=***light-reflecting sources***.(walls, mirror)
- Luminous object can be both a light source and a light-reflecting source (plastic globe with a bulb inside)

- **A** luminous object, in general, can be both a light source and a light reflector.



1.Point Light Source

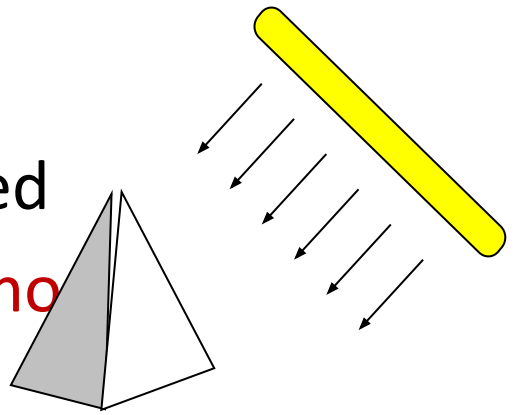
- The rays emitted from a point light radially diverge from the source.
- Approximation for sources that are small compared to the size of objects in the scene.
- Sources sufficiently far from scene (sun)
- A point light source is a fair approximation to a local light source such as a light bulb.
- Specified with its position(P)and intensity (color) of emitted light (I)
- The direction of the light to each point on a surface changes when a point light source is used.



Diverging ray paths from a point light source.

2. Distributed Light Source

- A nearby source, such as long fluorescent light.
- All of the rays from a directional/distributed light source have the same direction, and no point of origin.
- It is as if the light source was infinitely far away from the surface that it is illuminating.
- As area of source is not comparably small, approximation wont be realistic, so model should consider accumulated illumination effects of points over the source surface



An object illuminated with a distributed light source.

3. Reflection of light/ Material

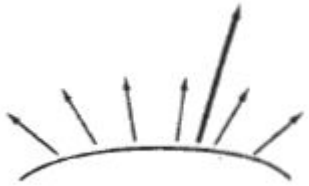
- When light is incident on an opaque surface, part of it is reflected and part is absorbed.

$$I = A + R$$

- The amount of incident light reflected by a surface depends on the type of the material.
- Shiny materials reflect more of the incident light
- Dull surface absorb more of the incident light.
- For an illuminated transparent surface, some of the incident light will be reflected and some will be transmitted through the material.

4. Distributed light source

- Surfaces that are rough, or grainy, tend to scatter the reflected light in all directions.
- This scattered light is **called diffuse reflection**.
- A very rough matte surface produces primarily diffuse reflections, so that the surface appears equally bright from all viewing directions.
- In addition to diffuse reflection, light sources create highlights, or bright spots, called **specular reflection**.
- This highlighting effect is more pronounced on shiny surfaces than on dull surfaces.



BASIC ILLUMINATION MODELS

- Illumination models are used to calculate light intensities that we should see at a given point on the surface of an object.
- Lighting calculations are based on the optical properties of surfaces, the background lighting conditions, and the light-source specifications.
- All light sources are considered to be point sources, specified with a coordinate position and an intensity value (color).

Basic Illumination Models

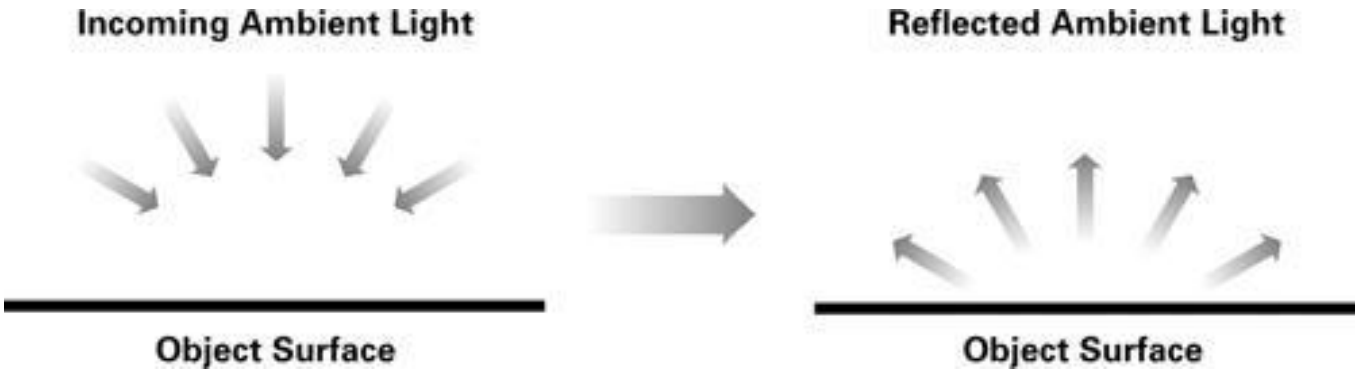
Lighting calculations are based on:

- Optical properties of surfaces(glossy, matte, opaque, and transparent). This controls the amount of reflection and absorption of incident light.
- Background lighting conditions.
- Light-source specifications- All light sources are considered to be point sources, specified with a coordinate position and intensity value (color).

Ambient Light

- This is a simplest illumination model.
- The equal amount of light from all directions
- A surface that is not exposed directly to a light source still will be visible it nearby objects are illuminated.
- This is a simple way to model the combination of light reflections from various surfaces to produce a uniform illumination called the **ambient light, or background light**.
- Ambient light has no spatial or directional characteristics.

- The amount of ambient light incident on each object is a constant for all surfaces and over all directions.
- The amount of ambient light that is reflected by an object is **independent of the objects position or orientation** and depends only on the optical properties of the surface.



- The level of ambient light in a scene is a parameter I_a, and each surface illuminated with this constant value.
- Illumination equation for ambient light is

$$I = k_a I_a$$

where

I is the resulting intensity

I_a is the incident ambient light intensity

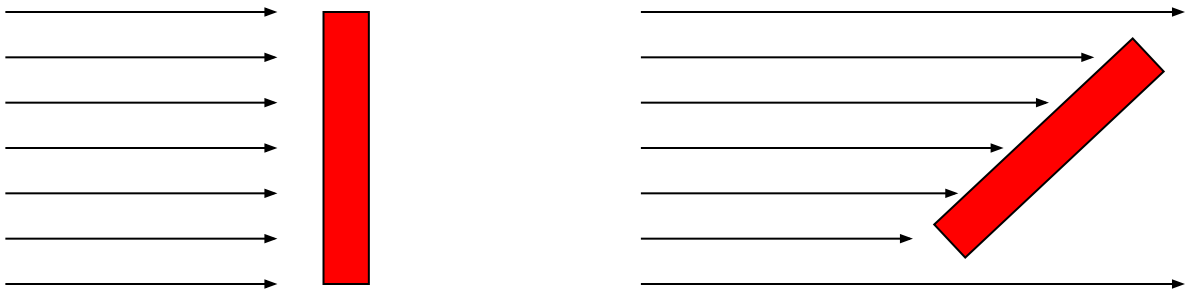
k_a is the object's basic intensity, ***ambient - reflection coefficient***.

Features of Ambient lighting

- Light from the environment
- Light reflect or scattered from other objects
- Coming uniformly from all direction and then reflected equally to all direction

Diffuse Reflection

- Even though there is equal light scattering in all direction from a surface, the brightness of the surface does depend on the **orientation of the surface** relative to the light source:
- The reflected light is **independent of viewing position**



(a) (b)

A surface perpendicular to the direction of the incident light (a) is more illuminated than an equal-sized surface at an oblique angle (b) to the incoming light direction

Diffuse Reflection

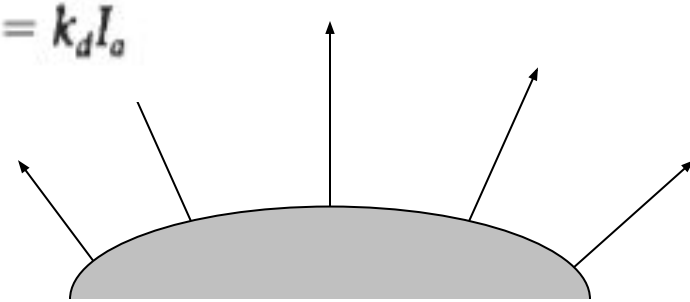
- Grainy surfaces scatter the reflected light in all directions. This scattered light is called ***diffuse reflection***.
- The surface appears equally bright from all viewing directions. this type of surface are called ideal diffuse reflection or Lambertian reflection
- What we call the color of an object is the color of the diffuse reflection of the incident light.
- Diffuse reflections are constant over each surface in a scene, independent of the viewing direction.
- The fractional amount of the incident light that is diffusely reflected can be set for each surface with parameter k_d (***diffuse-reflection coefficient***, or ***diffuse reflectivity***)

  $\mathbf{0 \le k_d \le 1;}$

  $k_d \sim 1$ – highly reflective surface;

  $k_d \sim 0$ – surface that absorbs most of the incident light;

$$I_{ambdiff} = k_d I_a$$



Diffuse reflection from a surface

Diffuse Reflection

Figure a) illustrates the illumination with diffuse reflection, using various values of parameter k_d between 0 and1.

Fig. a

Series of pictures of sphere illuminated by diffuse reflection model only using different k_d values (0.4, 0.55, 0.7, 0.85,1.0)

Combining the ambient and point-source intensity calculations to obtain an expression for the total diffuse reflection, where both k_a and k_d depend on surface material properties and are assigned values in the range from 0 to 1

$$I_{diff} = k_a I_a + k_d I_l (\mathbf{N} \cdot \mathbf{L})$$

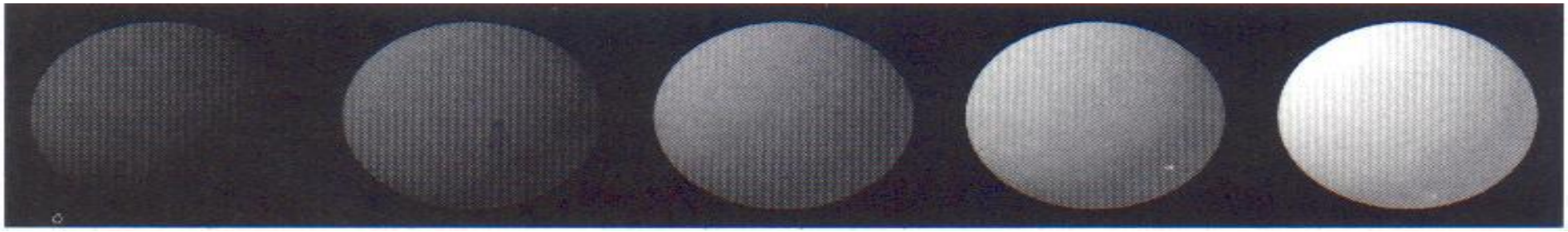


Fig. b

Series of pictures of sphere illuminated by ambient and diffuse reflection model.
$I_a = I_l = 1.0$, $k_d = 0.4$ and k_a values (0.0, 0.15, 0.30, 0.45,0.60).

The intensity of diffuse reflection due to ambient light is;

$$I_{adiff} = k_a I_a$$

The radiant energy from any small surface dA in any direction relative to surface normal is proportional to cos θ. That is, brightness depends only on the angle θ between the light direction L and the surface normal N.

∴ Light intensity $\alpha \cos \theta$.

If I_l is the intensity of the point light source and k_d is the diffuse reflection coefficient, then the diffuse reflection for single point-source can be written as;

$$I_{pdiff} = k_d I_l \cos\theta$$
$$I_{pdiff} = k_d I_l (N \cdot L)$$

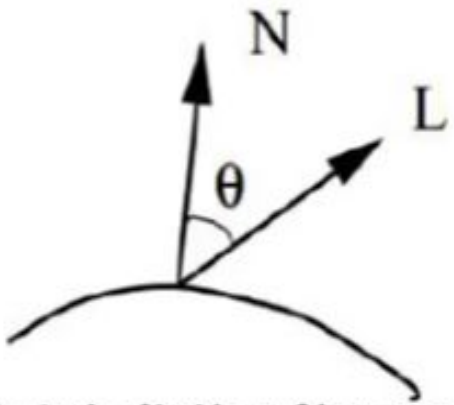


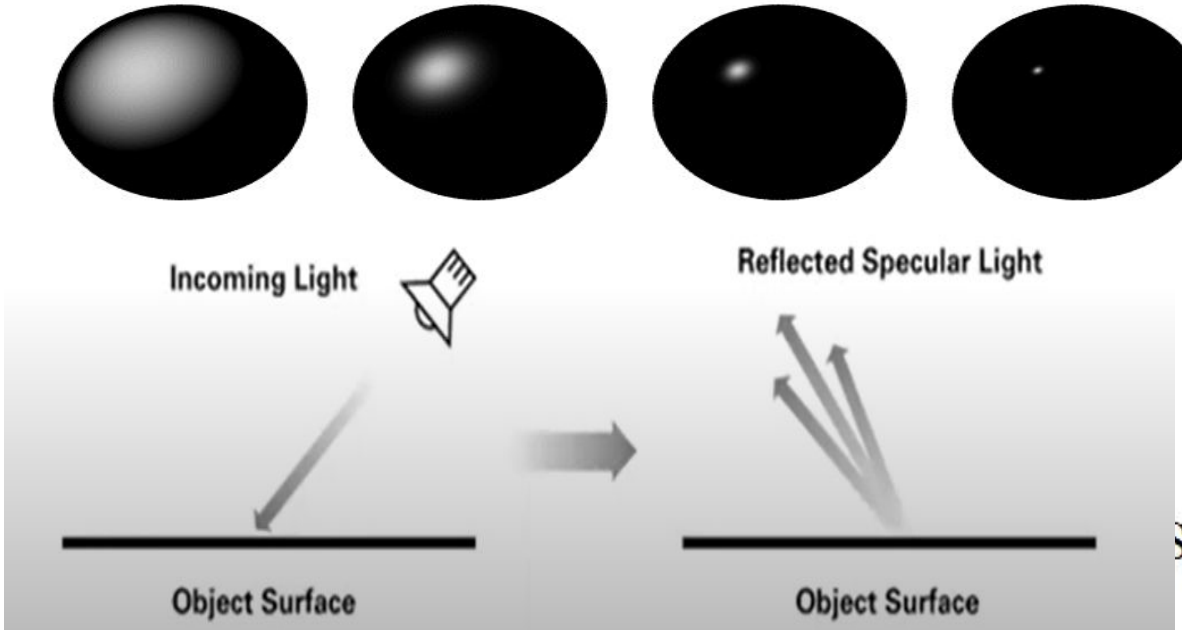
Fig: Angle of incidence θ between the unit light-source direction vector L and the unit surface normal

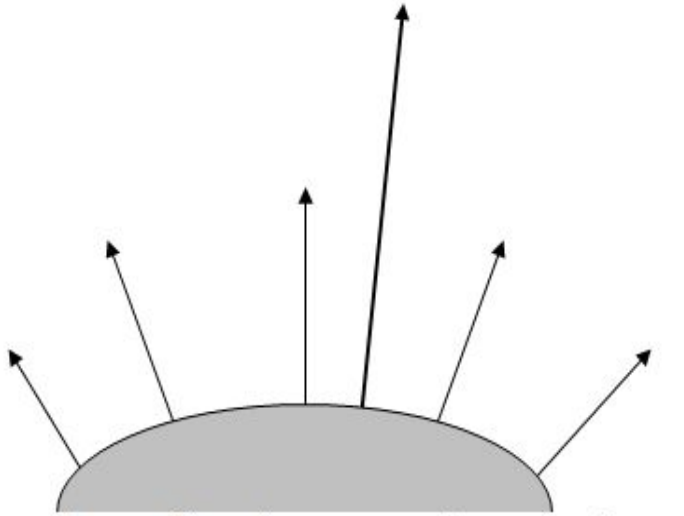
∴ Total diffuse reflection (Idiff) = Diff (due to ambient light)+Diff. due to pt.source.

$$I_{diff} = I_{adiff} + I_{pdiff}$$
$$I_{diff} = k_a I_a + k_d I_l (N \cdot L)$$

Specular Reflection

- Light sources create highlights, bright spots, called ***specular reflection***.
- More pronounced on shiny surfaces than on dull. (polished metal, apple, forehead, bright spot, etc)
- ***Specular reflection*** is the result of total, or near total, reflection of the incident light in a concentrated region around the ***specular-reflection angle.*** Shiny surfaces have narrow specular-reflection range and dull surfaces have a wider specular-reflection range.





Specular reflection superimposed on diffuse reflection vectors.

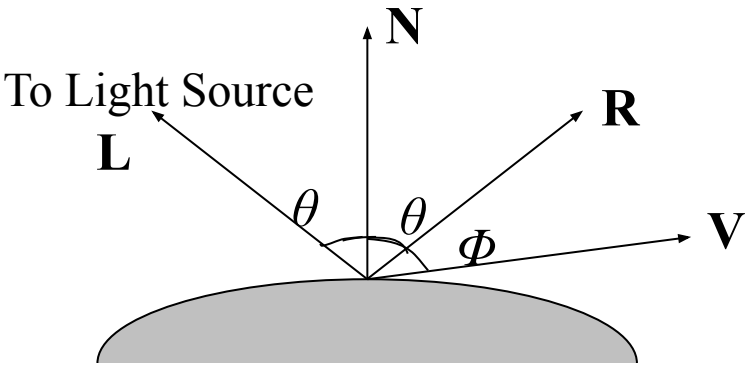
Specular Reflection…

- Depends on **surface orientation and viewer location**.

Figure: Specular -reflection direction at a point on illuminated surface

- **R -** represents the unit vector in the direction of specular reflection;
- **L -** unit vector directed toward the point light source;
- **V -** unit vector pointing to the viewer from the surface position;
- ϕ - viewing angle relative to the specular-reflection direction **R**.
- θ - angle of incident light =Specular -reflection angle

Note: For ideal reflector (perfect mirror), incident light reflected only in Specular -reflection direction. V and R coincide **(ϕ = 0)**



Modeling specular reflection.

Phong Specular-Reflection Model

Phong model is an empirical model for calculating specular-reflection range:

- Sets the intensity of specular reflection proportional to $\cos^{ns}\Phi$; (ns= n_s)
- Angle Φ assigned values in the range 0^o to 90^o, so that $\cos\Phi$ values from 0 to 1;
- ***Specular-reflection parameter n_s*** is determined by the type of surface,
- ***Specular-reflection coefficient k_s*** equal to some value in the range 0 to 1 for each surface.

(***k_s ~ $W(\theta)$, Specular-reflection function, Fresnel's law of reflection)***

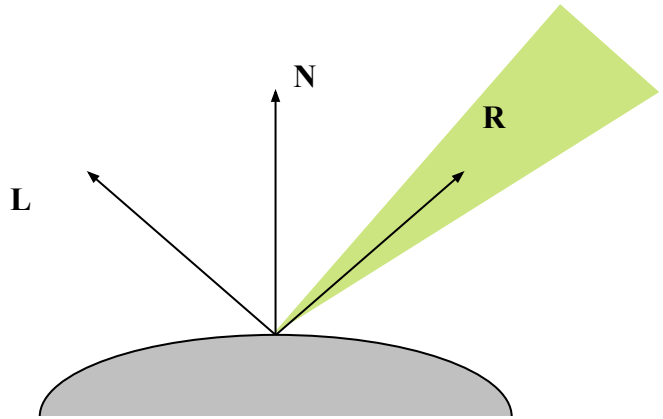
- ***Φ*** - viewing angle relative to the specular-reflection direction **R**.
- ***I_l - intensity of light source***

$$I_{spec} = W(\theta) I_l \cos^{ns}\Phi, \quad \textit{replace } W(\theta) \textit{ by } k_s$$

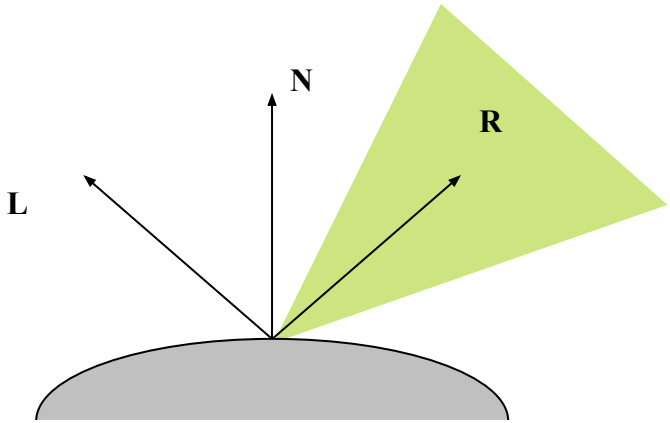
$$\boldsymbol{I_{spec} = k_s I_l \cos^{ns}\Phi}$$

Phong Model

- Very shiny surface is modeled with a large value for n_s
- Small values are used for duller surfaces.
- For perfect reflector (perfect mirror), n_s is infinite;

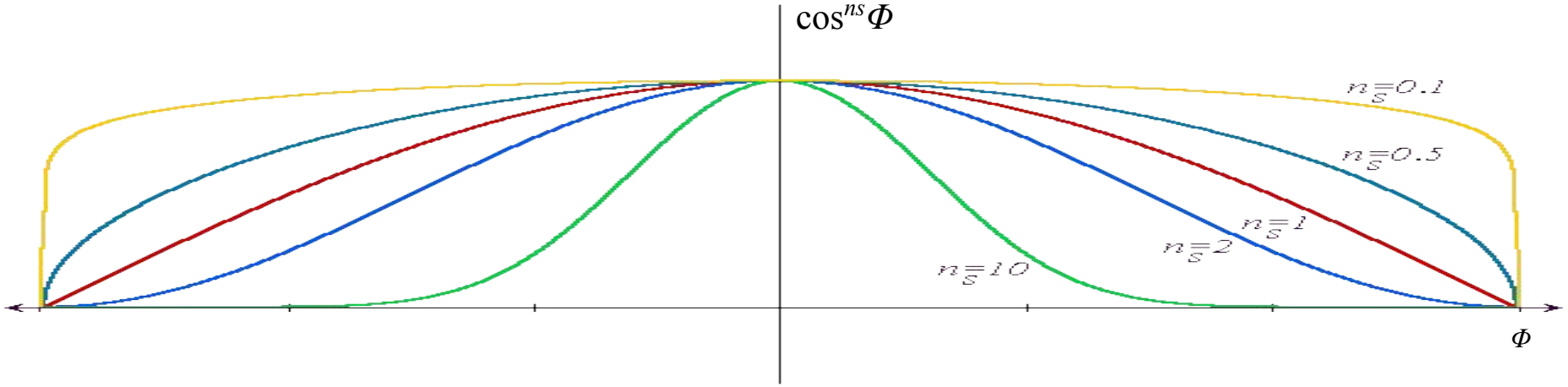


Shiny Surface (Large n_s)



Dull Surface (Small n_s)

Modeling specular reflection with parameter n_s.



Plots of $\cos^{ns}\Phi$ for several values of specular parameter n_s.

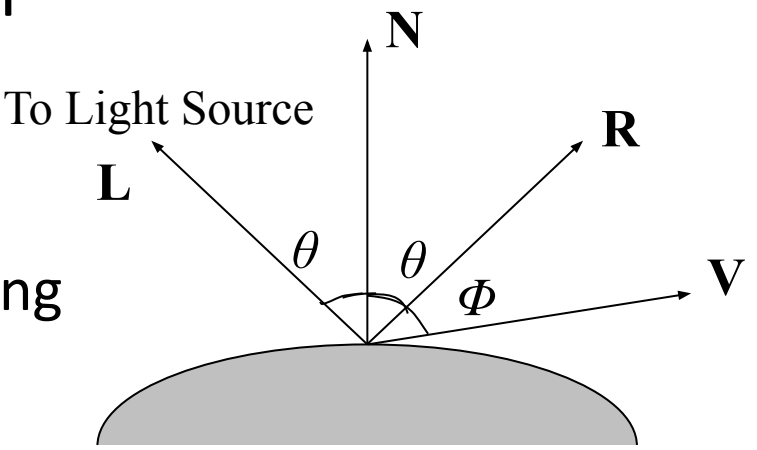
Phong Model

Phong specular-reflection model:

$$I_{spec} = k_s I_l \cos^{ns}\Phi$$

Since **V** and **R** are unit vectors in the viewing and specular-reflection directions, $\cos^{ns}\Phi$ calculated with the dot product **V**·**R**.

$$I_{spec} = k_s I_l (\mathbf{V}\cdot\mathbf{R})^{ns}$$

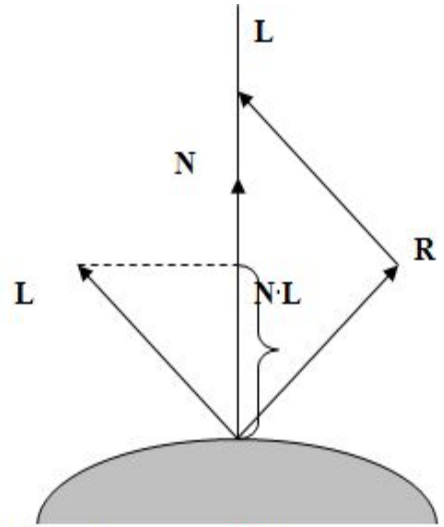


Modeling specular reflection.

Specular reflection vector R can be calculated in terms of vectors **N** and **L**. Projection of **L** onto direction of **N** is got with dot product **N.L.**

$$\mathbf{R} + \mathbf{L} = (2\ \mathbf{N}\cdot\mathbf{L})\mathbf{N}$$

$$\mathbf{R} = (2\ \mathbf{N}\cdot\mathbf{L})\mathbf{N} - \mathbf{L}$$



Calculation of vector **R** by considering projections onto the direction of the normal vector **N**.

Phong Model

Simplified Phong model use half way vector **H,** between **L** and **V** to find range of specular reflections. If **V**·**R** is replaced with dot product **N**·**H,** it simply replaces empirical cosΦ calculation with empirical cosα calculation. Half way vector **H** is got by:

$$\mathbf{H} = (\mathbf{L} + \mathbf{V}) / |(\mathbf{L} + \mathbf{V})|$$

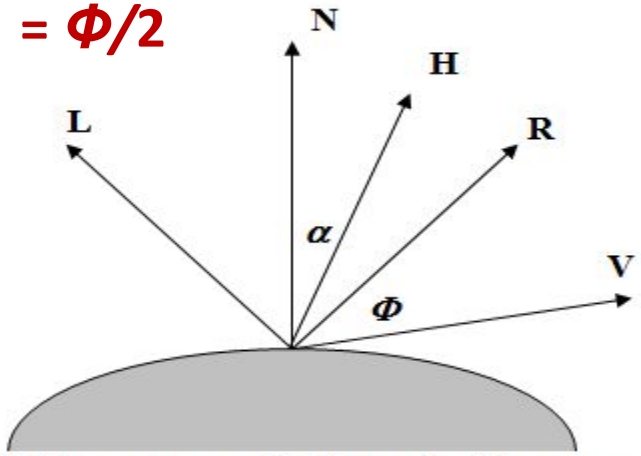
If both viewer and light source very far from surface, **L,V** and **H** are constant over surface. For Non-planar surfaces, **N**·**H** needs less computation since calculation of **R** involves **N**. For a given light source and viewer positions, vector **H** is **orientation direction** for surface that would produce **maximum specular reflection (highlights).**

If vector **V** is **co-planar** with **L** and **R (**and thus **N),** $\alpha = \Phi/2$

If vector **V**, **L** and **N** is **not co-planar,** $\alpha > \Phi/2$

depending on spatial relationship of these vectors

$$I_{spec} = k_s I_l (\mathbf{N}\cdot\mathbf{H})^{ns}$$



Halfway vector **H** along the bisector of the angle between **L** and **V**.

Combine Diffuse & Specular Reflections

For a single point light source, the combined diffuse and specular reflections from a point on an illuminated surface modelled as:

$$I = I_{diff} + I_{spec}$$
$$= k_a I_a + k_d I_l (\mathbf{N} \cdot \mathbf{L}) + k_s I_l (\mathbf{N} \cdot \mathbf{H})^{ns}$$

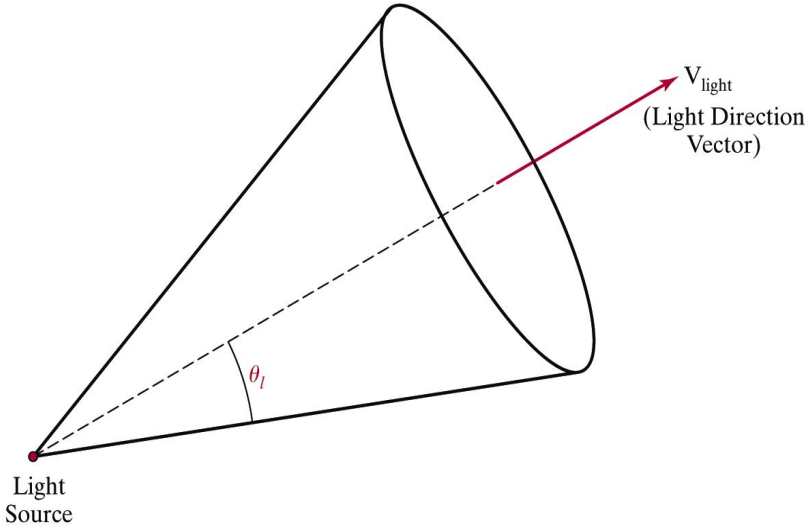
If multiple light sources placed, light reflection at any surface point is obtained by summing the contributions from the individual sources as:

$$I = k_a I_a + \sum_{i=1}^{n} I_{li} [k_d (\mathbf{N} \cdot \mathbf{L}_i) + k_s (\mathbf{N} \cdot \mathbf{H}_i)^{ns}]$$

Warn Model

– Warn Model -Simulates studio lighting effects by controlling light intensity in different directions.

- Then the intensity in different directions is controlled by selecting values for the Phong exponent
- light controls, such as "barn doors" and spotlighting, used by studio photographers can be simulated in the Warn model.
- Flaps are used to control the amount of light emitted by a source in various directions



- Two flaps are provided for each of the **x,** y, and ***z*** directions.
- Spotlights are used to control the amount of light emitted within a cone with apex at a point-source position.

Intensity Attenuation

- As radiant energy from a point light source travels through space, its amplitude is attenuated by the factor **$1/d^2$**
- Closer surface gets higher incident intensity from source

Sometimes, more realistic attenuation effects can be obtained with an inverse quadratic function of distance

$$f = \begin{cases} 1.0 & \text{if source is at infinity} \\ \dfrac{1}{a_0 + a_1 d + a_2 d^2} & \text{if source is local} \end{cases}$$

The intensity attenuation is not applied to light sources at infinity because all points in the scene are at a nearly equal distance from a far-off source

Intensity Attenuation

- With a given set of attenuation coefficients, magnitude of attenuation function can be limited to 1 with the calculation

$$f(d) = \min(1, \frac{1}{a_0 + a_1 d + a_2 d^2})$$

- Using this function, basic illumination model is:

$$\boldsymbol{I = k_a I_a + \sum_{i=1}^{n} f(d_i) I_{li} [k_d (N \cdot L_i) + k_s (N \cdot H_i)^{ns}]}$$

where d_i is distance light has traveled from light source

Color Considerations

- To write the intensity equation as a function of the color properties of the light **sources** and object surface.
- For a RGB color description, each color in a scene is expressed in terms of Red, Green and Blue components
- One way to set surface colors is by specifying the reflectivity coefficients as three-element vectors
- Diffuse reflection coefficient vector - RGB components (k_{dR}, k_{dG}, k_{dB})
- For a RGB color description, each intensity and reflectance specification is a three-element vector k_a, k_d, k_s
- The sum of reflectance coefficients is $k_a + k_d + k_s \leq 1$

For a blue surface, (k_{dB} - non-zero , k_{dR}, k_{dG} - zero) intensity is:

$$I_B = k_{aB} I_{aB} + \sum_{i=1}^{n} f(d_i) I_{lBi} [k_{dB}(N \cdot L_i) + k_{sB}(N \cdot H_i)^{ns}]$$

- Surfaces typically are illuminated with white light sources, and in general we can set surface color so that the reflected light has nonzero values for all three RGB components.
- Calculated intensity levels for each color component can be **used** to adjust the corresponding electron gun in an RGB monitor.
- Another method for setting surface color is to specify the components of diffuse and specular color vector for each surface, while retaining the reflectivity coefficients as single-valued constants

Using components of two surface color vectors (S_{dR}, S_{dG}, S_{dB}) and (S_{sR}, S_{sG}, S_{sB}) blue component of reflected light is:

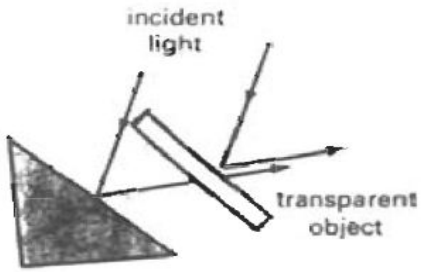
$$I_B = k_a S_{dB} I_{aB} + \sum_{i=1}^{n} f(d_i) I_{lBi} [k_d S_{dB} (N \cdot L_i) + k_s S_{dB} (N \cdot H_i)^{ns}]$$

Any component of a color specification can be represented with its spectral wavelength λ, intensity is:

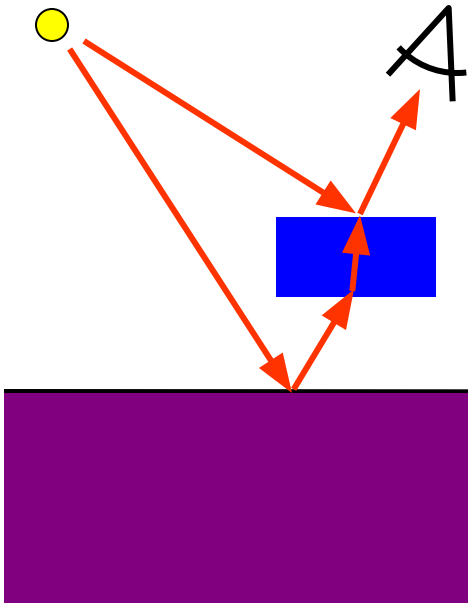
$$I_\lambda = k_a S_{d\lambda} I_{a\lambda} + \sum_{i=1}^{n} f(d_i) I_{l\lambda i} [k_d S_{d\lambda} (N \cdot L_i) + k_s S_{d\lambda} (N \cdot H_i)^{ns}].$$

Transparency

- A transparent surface, in general, produces both reflected and transmitted light. The relative contribution of the transmitted light depends on the degree of transparency of the surface and whether any light sources or illuminated surfaces are behind the transparent surface.
- When a transparent surface is to be modeled, the intensity equations must be modified to include contributions from light passing through the surface.
- usually the transmitted light is generated from reflecting objects in back of the surface. Realistic transparency effects are modeled by considering light refraction.
- When light is incident upon a transparent surface, part of it is reflected and part is refracted.



Transparency 1

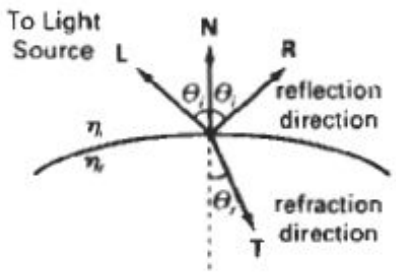


Transparent object:

- reflected and transmitted light
- refraction
- scattering

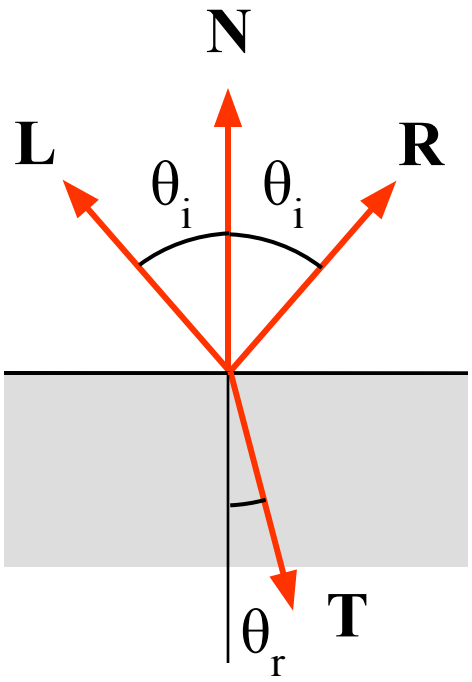
- Because the speed of light is different in different materials, the path of the refracted light is different from that of the incident light.
- The direction of the refracted light specified by the angle of refraction, is a function of the index of refraction of each material and the direction of the incident light.
- Angle of refraction θ_r , is calculated from the angle of incidence θ_i , the index of refraction η_i of the "incident" material (usually air), and the index of refraction η_r , of the refracting material according to *Snell's law:*

Transparency 2



Snell's law of refraction:

$$\sin\theta_r = \frac{\eta_i}{\eta_r}\sin\theta_i,\ \eta : \text{index of refraction}$$



- The index of refraction of a material is a function of the wavelength of the incident Light, so that we different color components of a Light ray will be refracted at different angles.
- For most applications, we can use an average index of refraction for the different materials that are modeled *in* a scene. The index of refraction of air is approximately 1, and that of crown glass is about 1.5.
- the unit transmission vector T in the refraction direction θ_r , as

$$\mathbf{T} = \left(\frac{\eta_i}{\eta_r} \cos\theta_i - \cos\theta_r \right) \mathbf{N} - \frac{\eta_i}{\eta_r} \mathbf{L}$$

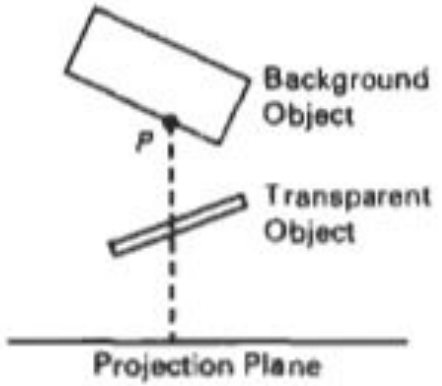
- where N is the unit surface normal, and L is the unit vector in the direction of the light source

- **A** simpler procedure for modeling transparent objects is to ignore the path shifts altogether.
- In effect, this approach assumes there is no change in the index of refraction from one material to another, so that the angle of refraction is always the same as the angle of incidence.
- This method speeds up the calculation of intensities and can produce reasonable transparency effects furter hin polygon surfaces.

- We can combine the transmitted intensity I_{trans} through a surface from a background object with the reflected intensity I_{refl} from the transparent surface using a transparency coefficient k_t.
- We assign parameter k_t, a value between 0 and 1 to specify how much of the background light is to be transmitted.
- Total surface intensity is then calculated as

$$I = (1 - k_t)I_{refl} + k_tI_{trans}$$

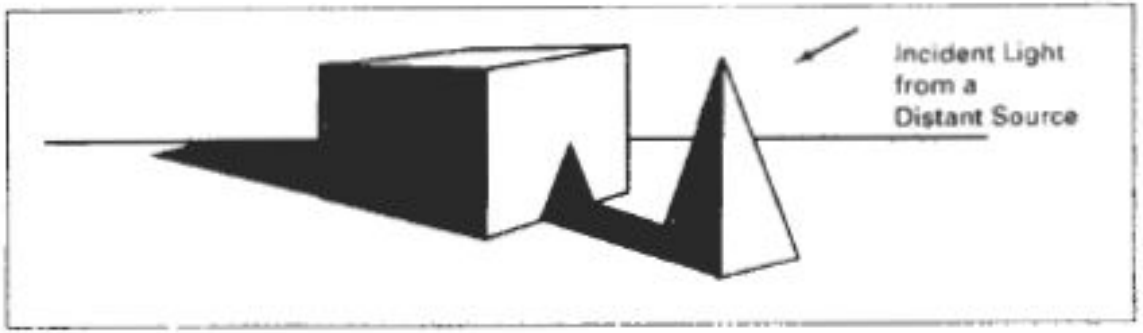
- The term **(1** - k,) is the opacity factor.



- Transparency effects are often implemented with modified depth-buffer **(z**buffer) algorithms.
- A simple way to do this is to process opaque objects first to determine depths for the visible opaque surfaces.
- Then, the depth positions of the transparent objects are compared to the values previously stored in the depth buffer.
- If any transparent surface is visible, its reflected intensity is calculated and combined with the opaque-surface intensity previously stored in the frame buffer.
- This method can be modified to produce more accurate displays by using additional storage for the depth and other parameters of the transparent surfaces.

Shadow

- Shadow can help to create realism. Without it, a cup, eg., on a table may look as if the cup is floating in the air above the table.
- By applying hidden-surface methods with a light source at the viewing position, identify which surface sections cannot be "seen" from the light source => shadow areas.
- shadow areas for all light sources found, shadows could be treated as surface patterns and stored in pattern arrays
- usually shadow areas displayed with ambient-light intensity only or combined with surface textures.



Polygon -Rendering Methods

- Realistic displays of a scene are obtained by generating perspective projections of objects and by applying natural lighting effects to the visible surface.
- Intensity calculation from an illumination model can be applied to surface rendering in various ways.
- We could use an illumination model to determine the surface intensity at every projected pixel position or we could apply the illumination model to a few selected points and approximate the intensity at the other surface position.

- Scan line algorithms typically apply a lighting model to obtain polygon surface rendering in one of two ways.
- Each polygon can be rendered with a single intensity, or the intensity can be obtained at each point of the surface using an interpolation scheme.
  - Flat Surface Rendering
  - Gouraud Surface Rendering
  - Phong Surface Rendering

Flat Surface Rendering

- The simplest method for rendering a polygon surface

Constant-intensity rendering or flat surface rendering:

- Determine color for center of polygon by using face or one normal from a pair of edges
- Fill the polygon with a constant color.

To provide an accurate rendering, following assumptions are valid:

- The object is a polyhedron and is not an approximation of an object with curved surface.
- All light source illuminating the object are sufficiently far from the surface so that N·L and the attenuation function are constant over the surface.
- The viewing position is sufficiently far from the surface so that V·R is constant over the surface.

Flat Surface Rendering

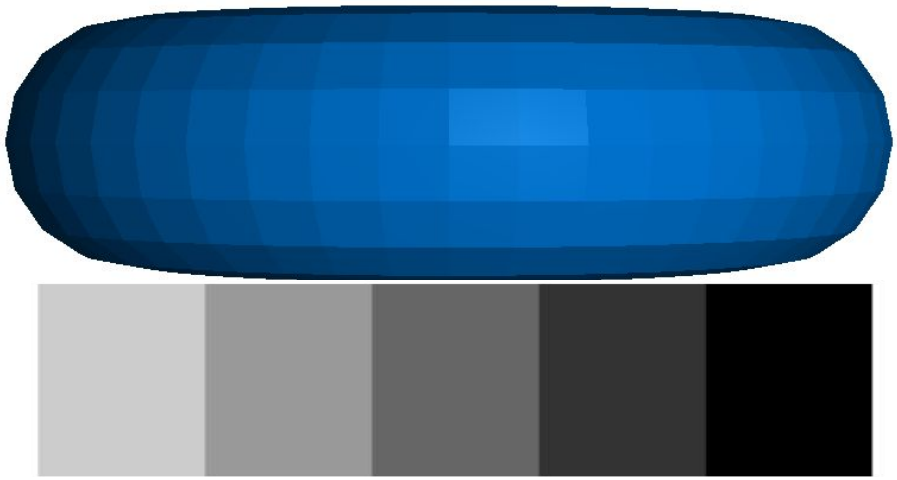
- It provides accurate rendering of an object if:
  - Object consists of planar faces, and
  - Light sources are far away, and
  - Eye point is far away, or
  - Polygons are about a pixel in size, or
  - if polygon facets approximate the surface well

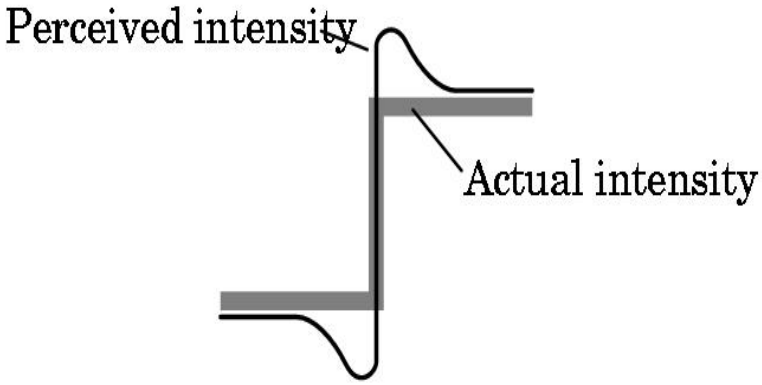
- Flat surface rendering is:
  - extremely fast and simple
  - but not realistic

Flat Surface Rendering

- Limitations
  - Highlights not visible,
  - Facetted appearance, increased by Mach banding effects
  - Not realistic , intensity discontinuities
- **Mach Banding**

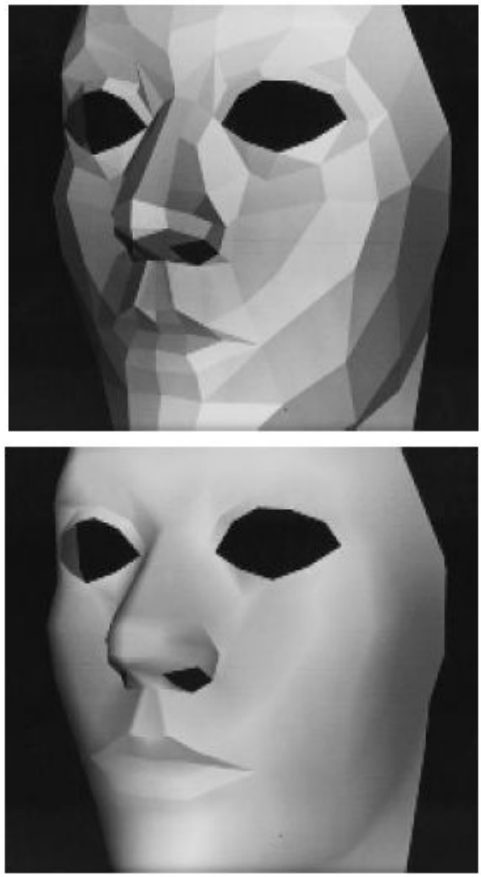
  Human perception: edges are given emphasis, contrast is increased near edges.





Gouraud Surface Rendering

- Gouraud surface shading developed in the 1970s by Henri Gouraud
- **known as intensity- interpolation surface rendering**
- Intensity levels are calculated at each vertex and linearly interpolated across the surface
- Intensity values for each polygon are matched with those of adjacent polygons along the common edges, eliminating intensity discontinuities of flat shading
- Gouraud surfacing rendering can be implemented relatively efficiently using an iterative approach
- Typically Gouraud shading is implemented as part of a visible surface detection technique



Gouraud Surface Rendering (cont…)

Steps of Gouraud surface rendering:

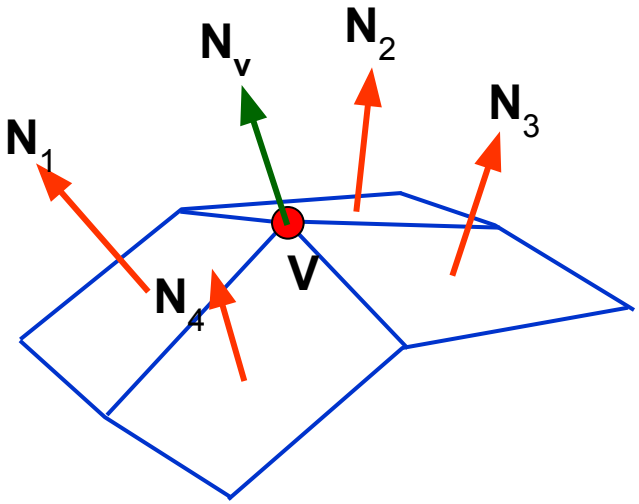
1. Determine the average unit normal vector at each vertex of the polygon
2. Apply an illumination model at each polygon vertex to obtain the light intensity at that position
3. Linearly interpolate the vertex intensities over the projected area of the polygon

Put Simply Gouraud shading:

- Determine average normal on vertices;
- Determine color for vertices;
- Interpolate the colors per polygon (incrementally)

Gouraud Surface Rendering (cont…)

At each polygon vertex, a normal vector is got by averaging the surface normals of all polygons sharing that vertex.



The average unit normal vector at v is given as:

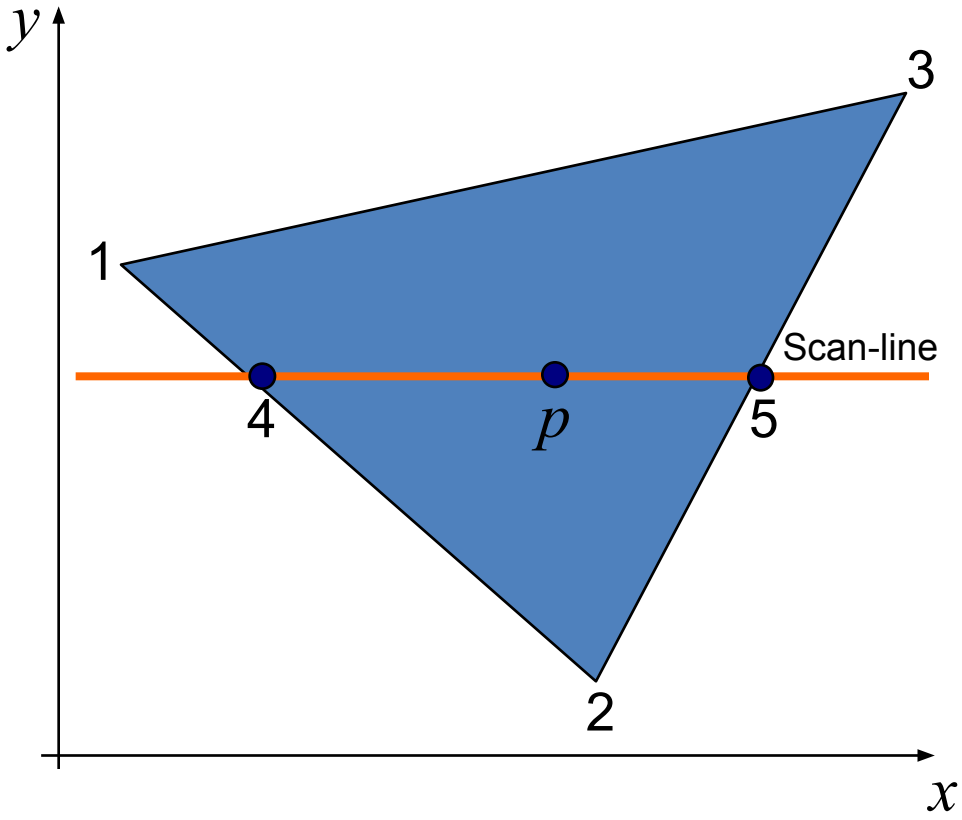
$$N_v = \frac{N_1 + N_2 + N_3 + N_4}{\left| N_1 + N_2 + N_3 + N_4 \right|}$$

or more generally:

$$N_v = \frac{\sum_{i=1}^{n} N_i}{\left| \sum_{i=1}^{n} N_i \right|}$$

Gouraud Surface Rendering (cont…)

- Illumination values are linearly interpolated across each scan-line from intensities at the edge points
  - I_4 and I_5 are interpolated along y-axis, and then
  - I_p is interpolated along x-axis



$$I_4 = \frac{y_4 - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y_4}{y_1 - y_2} I_2$$

$$I_5 = \frac{y_5 - y_2}{y_3 - y_2} I_3 + \frac{y_3 - y_5}{y_3 - y_2} I_2$$

$$I_p = \frac{x_5 - x_p}{x_5 - x_4} I_4 + \frac{x_p - x_4}{x_5 - x_4} I_5$$

Gouraud Surface Rendering (cont…)

Incremental Calculations also possible. They are used to obtain successive edge intensity values between scan lines and to obtain successive intensities along a scan line.

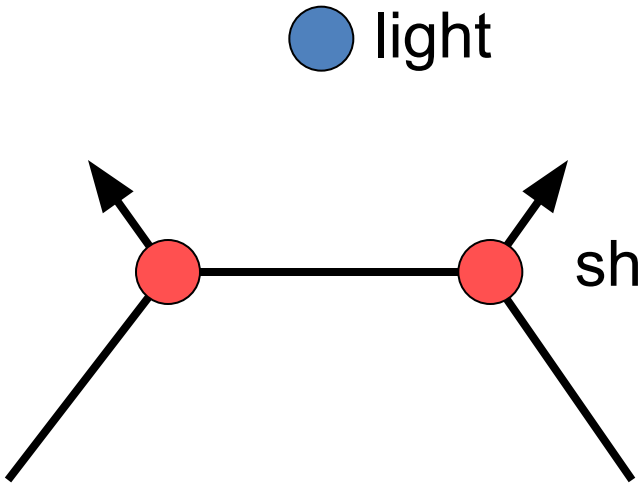
Intensity at edge position(x,y) is interpolated as:

$$I = \frac{y - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y}{y_1 - y_2} I_2$$

Intensity along this edge for the next scan line, y-1 is:

$$I' = I + \frac{I_2 - I_1}{y_1 - y_2}$$

Gouraud Interpolation Problems

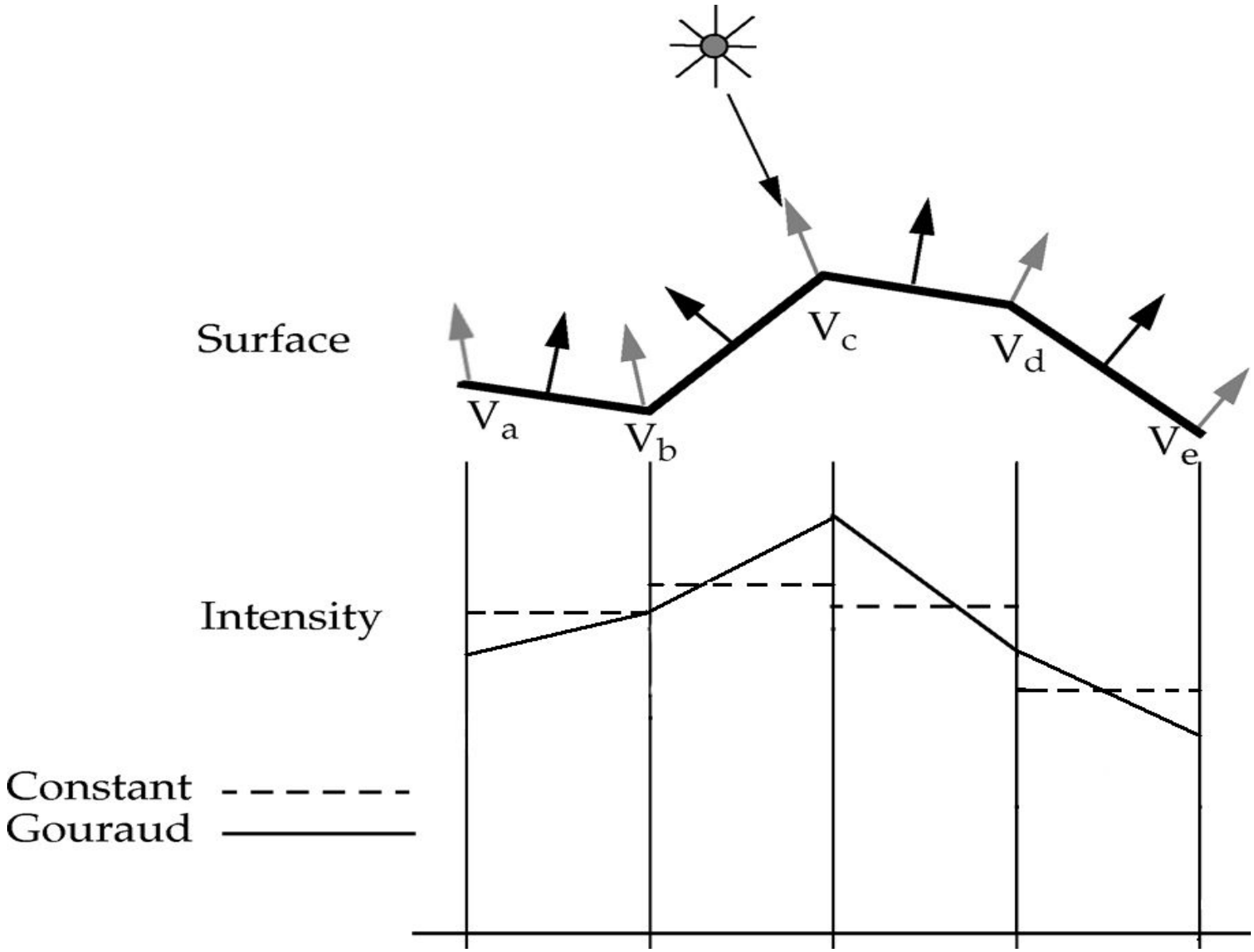


shading will be constant!

Misses some highlights

Shading is not linear

Gouraud Surface Rendering



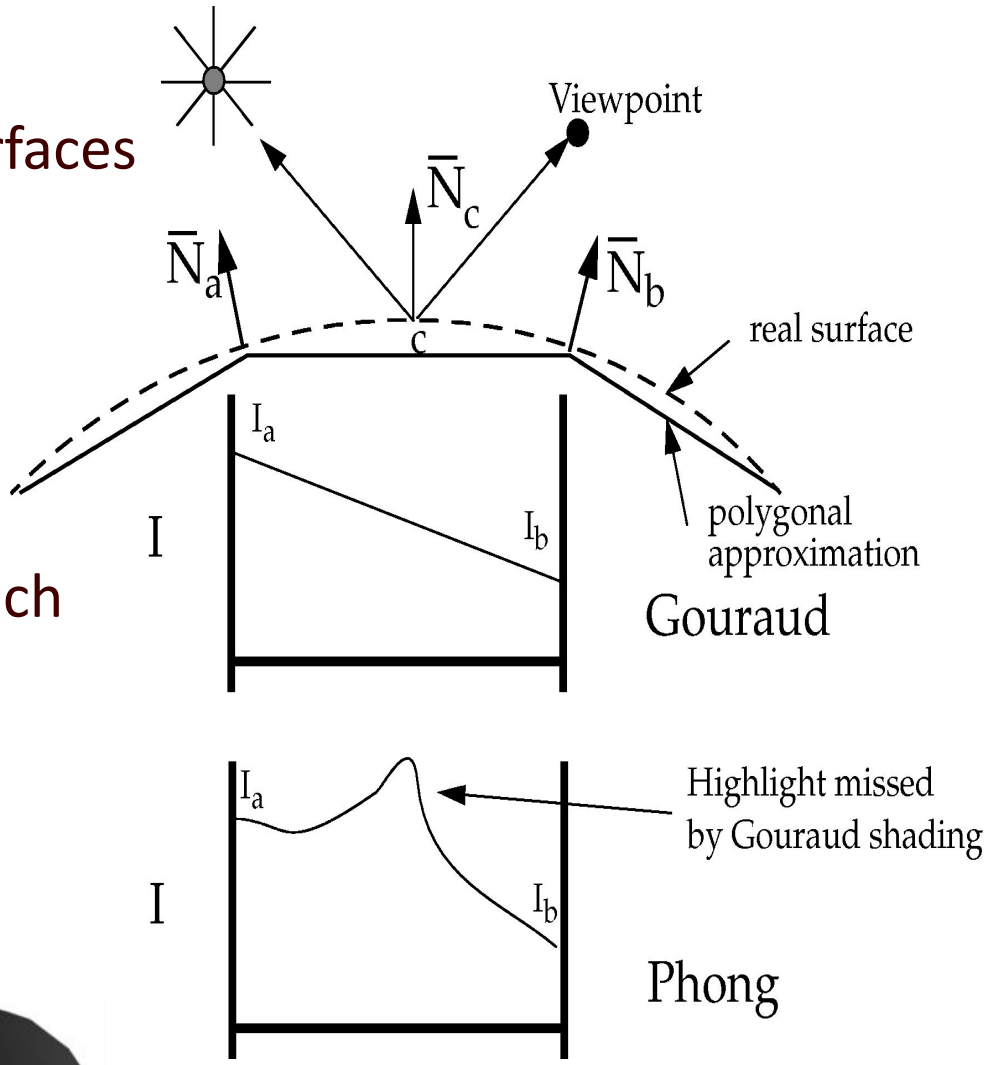
Gouraud Shading

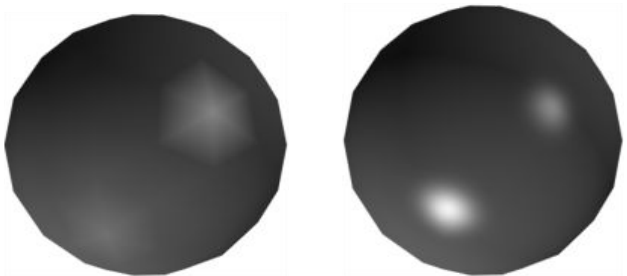
Advantages:

- Much better result for curved surfaces
- Avoid intensity discontinuities

Disadvantages:

- Errors near highlights
- Errors near specular reflections
- Linear interpolation still gives Mach banding
- Silhouettes are still not smooth
- Not so realistic





Phong Surface Rendering

- developed by **Phong Bui Tuong**
- Known as **normal-vector interpolation rendering-**
  interpolates normal vectors instead of intensity values
- A more accurate interpolation based approach
- Typically Phong shading is implemented as part of a
  visible surface detection technique

Phong Surface Rendering (cont…)

Steps of Phong surface rendering:

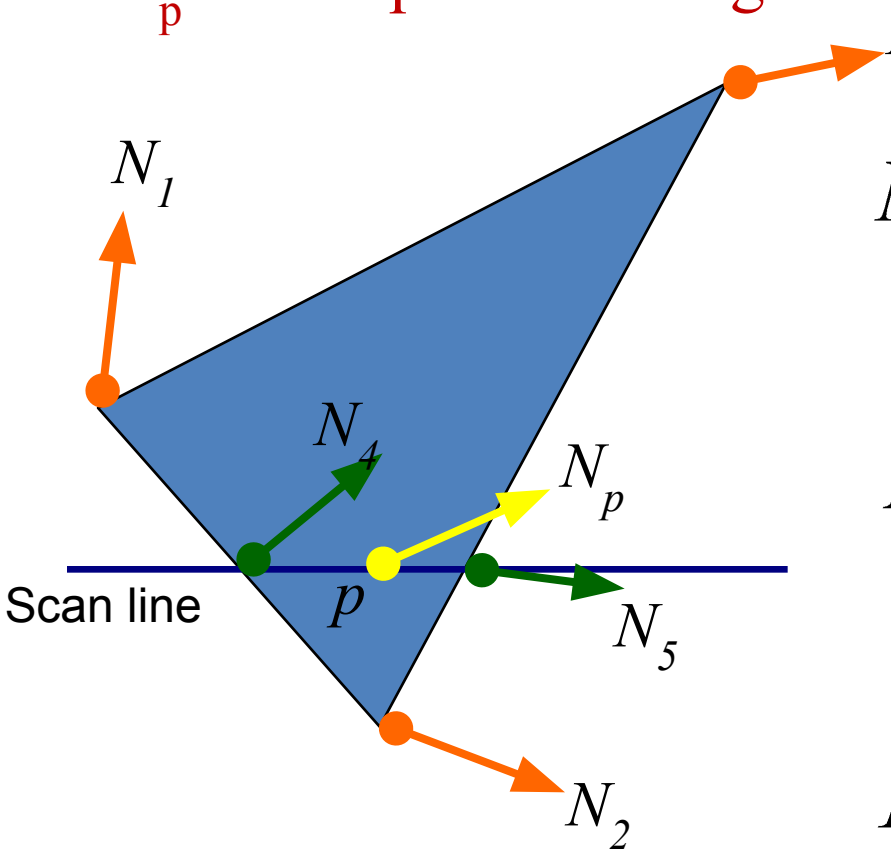
1. Determine the average unit normal vector at each vertex of the polygon
2. Linearly interpolate the vertex normals over the projected area of the polygon
3. Apply an illumination model at positions along scan lines to calculate pixel intensities using the interpolated normal vectors (per-pixel illumination)

Put simply Phong Shading:

- Determine average normal per vertex;
- Interpolate normals per polygon (incrementally);
- Calculate color per pixel.

Phong Surface Rendering (cont…)

- The normal vector N for each scan line intersection point obtained by vertically interpolating edge endpoint normals
  - N_4 and N_5 are interpolated along y-axis, and then
  - N_p is interpolated along x-axis



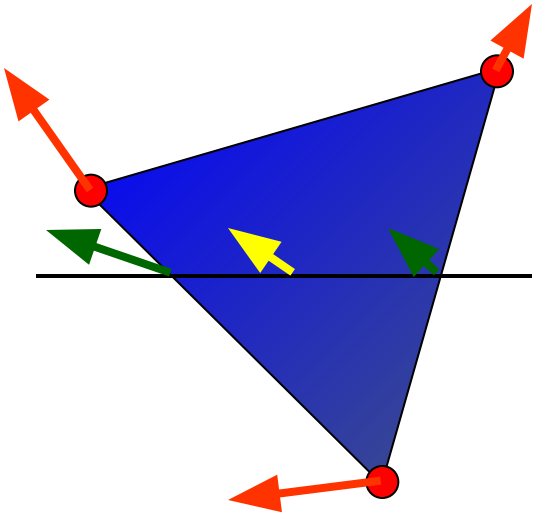
$$N_4 = \frac{y_4 - y_2}{y_1 - y_2} N_1 + \frac{y_1 - y_4}{y_1 - y_2} N_2$$

$$N_5 = \frac{y_5 - y_2}{y_3 - y_2} N_3 + \frac{y_3 - y_5}{y_3 - y_2} N_2$$

$$N_p = \frac{x_5 - x_p}{x_5 - x_4} N_4 + \frac{x_p - x_4}{x_5 - x_4} N_5$$

Phong Surface Rendering (cont…)

- The normal vector N for each scan line intersection point obtained by vertically interpolating edge endpoint normals
  - I_4 and I_5 are interpolated along y-axis, and then
  - I_p is interpolated along x-axis



$$N = \frac{y - y_2}{y_1 - y_2} N_1 + \frac{y_1 - y}{y_1 - y_2} N_2$$

Phong Surface Rendering

Advantages

- Even better result for curved surfaces
- No errors at high lights or specular reflections
- No Mach banding
- More Realistic

Disadvantages

- Silhouettes remain coarse
- More expensive than flat or Gouraud shading
- Much slower as the lighting model is revaluated so many times

- However, there are Fast Phong surface rendering approaches that can be implemented iteratively

Fast Phong Shading

- Fast Phong surface rendering approaches that can be implemented iteratively
- Like Phong surface rendering, but 2nd order approximation of color over polygon used
- Approximates intensity calculations using a Taylor-series expansion and triangular surface patches
- It can deal diffuse reflection, specular reflection
- It can deal with polygons other than triangles and finite viewing positions

- Since Phong shading interpolates normal vectors from vertex normal, we can express the surface normal N at any point ***(x,* y)** over a triangle as

$$\mathbf{N} = \mathbf{A}x + \mathbf{B}y + \mathbf{C}$$

- where vectors **A, B,** and **C** are determined from the three vertex equations:

$$\mathbf{N}_k = \mathbf{A}x_k + \mathbf{B}y_k + \mathbf{C}, \qquad k = 1, 2, 3$$

- write the calculation for light-source diffuse reflection from a surface point ***(x,* y)** as

$$I_{diff}(x, y) = \frac{\mathbf{L} \cdot \mathbf{N}}{|\mathbf{L}|\,|\mathbf{N}|}$$

$$= \frac{\mathbf{L} \cdot (\mathbf{A}x + \mathbf{B}y + \mathbf{C})}{|\mathbf{L}|\,|\mathbf{A}x + \mathbf{B}y + \mathbf{C}|}$$

$$= \frac{(\mathbf{L} \cdot \mathbf{A})x + (\mathbf{L} \cdot \mathbf{B})y + \mathbf{L} \cdot \mathbf{C}}{|\mathbf{L}|\,|\mathbf{A}x + \mathbf{B}y + \mathbf{C}|}$$

$$I_{diff}(x, y) = \frac{ax + by + c}{(dx^2 + exy + fy^2 + gx + hy + i)^{1/2}}$$

- where parameters such as *a,* b, c, and d are used to represent the various dot products.
- Finally, we can express the as a Taylor-series expansion and retain terms up to second degree in **x** and *y.*

$$I_{diff}(x, y) = T_5x^2 + T_4xy + T_3y^2 + T_2x + T_1y + T_0$$

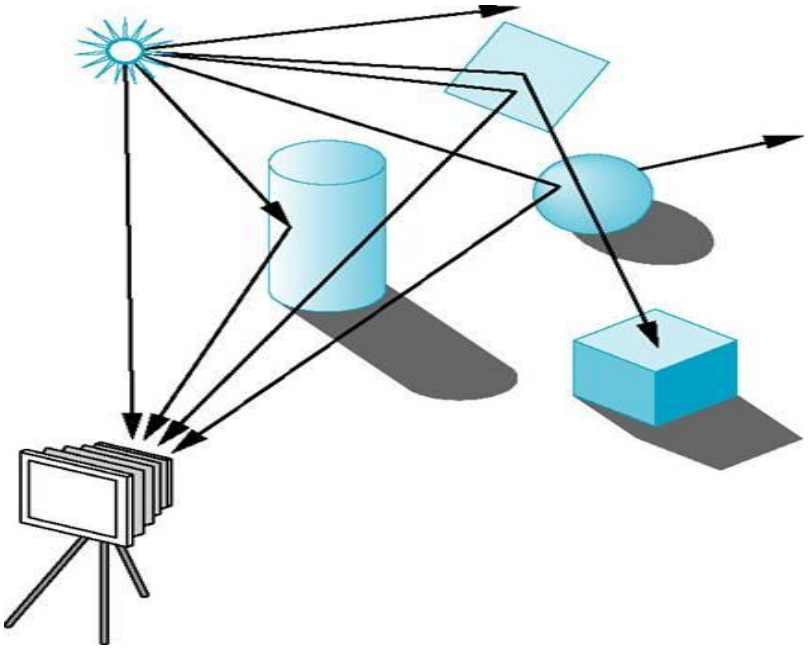
Ray **tracing**

- Ray **tracing is** an extension of this basic idea. Instead of merely looking for the visible surface for each pixel, we continue to bounce the ray around the scene, collecting intensity contributions.
- This provides a simple and powerful rendering technique for obtaining global reflection and transmission effects.
- The basic ray-tracing algorithm also provides for visible-surface detection, shadow effects, transparency, and multiple light-source illumination

Ray Tracing

- Extension to Ray casting

-Trace the multiple ray path to pick up the global reflection and refraction contribution from multiple objects in a scene.



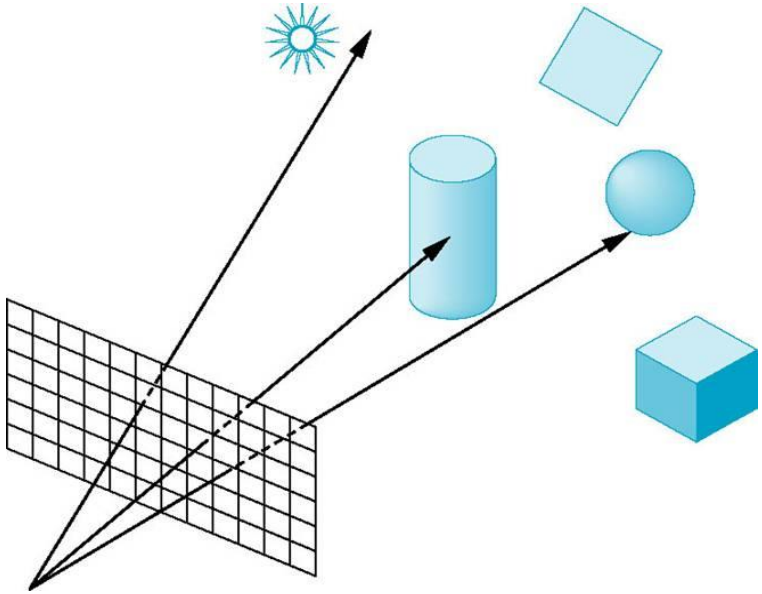
Two methods of ray tracing:

- Forward Ray Tracing

– Rays from light source bounce the objects before reaching the camera

- Backward Ray Tracing

– Track only those rays that finally made it to the camera



Basic Ray-Tracing Algorithm

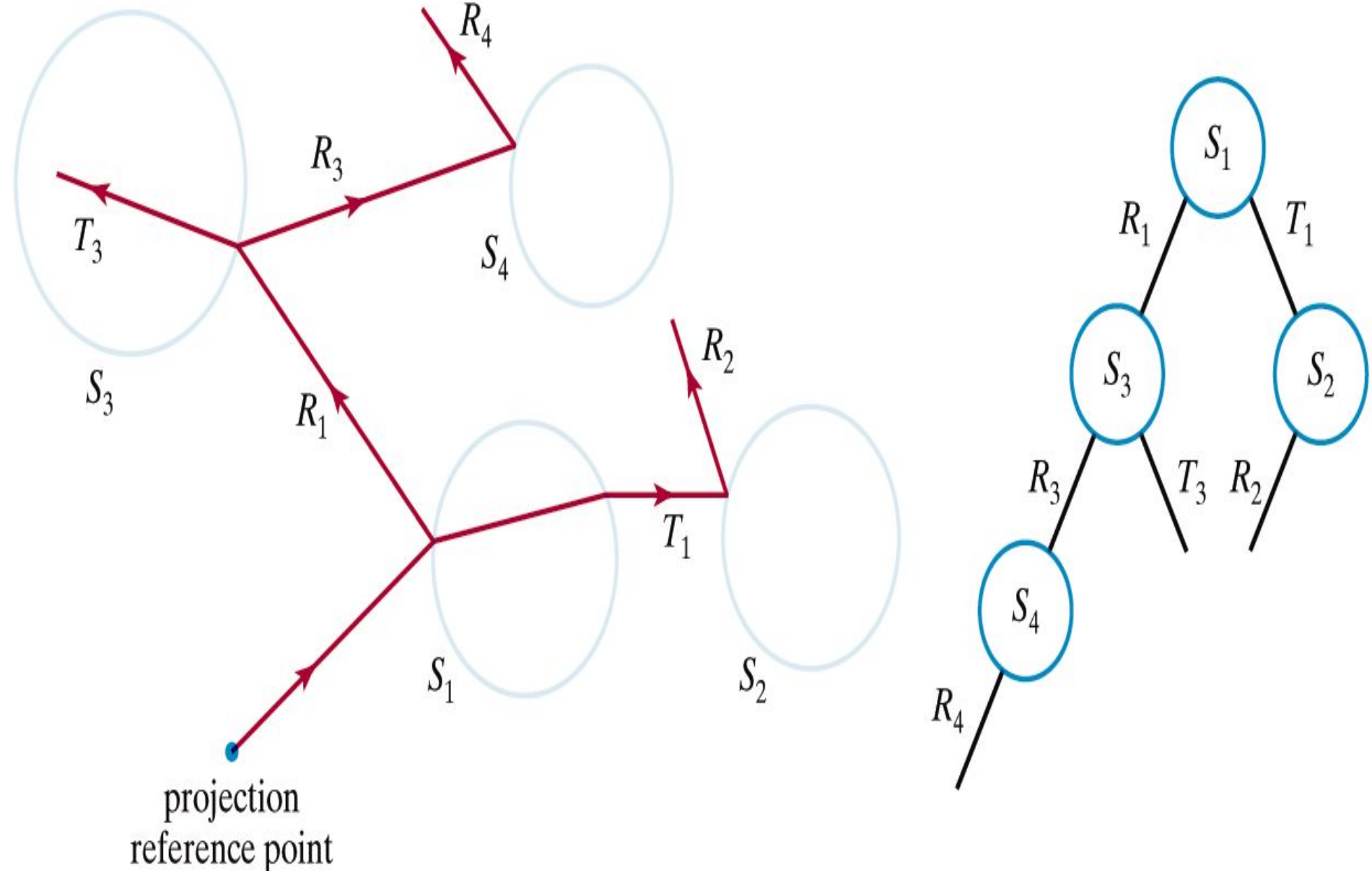
- Setup a coordinate system with pixel position in the xy plane.
- From the center of projection (COP), determine a ray that passes through the center of each pixel position.
- Determine all surfaces the ray intersects and order those by distance from the pixel. The nearest surface is the visible surface for that pixel.
- We then reflect the ray off the visible surface- along a specular path (angle of reflection equals angle of incidence)
- If the surface is transparent, we also send a ray through the surface in the retraction direction. Reflection and refraction rays are referred to as *secondary ray.*

- This procedure is repeated for each secondary ray: Objects are tested for intersection, and the nearest surface along a secondary ray path is used to recursively produce the next generation of reflection and refraction paths.
- Each successively intersected surface is added to a binary ray tracing tree.

•Fire off secondary rays from the surface that ray intersects.

- Shadow rays, *L :* Towards Light Sources (shadow feelers) to check occlusion / local illumination of diffuse surfaces
- Reflection rays, *R:* In reflection direction (specular reflection angle)
- Transmitted rays, *T:* In refraction direction by Snell's Law, of transparent surfaces

Ray Trace and Ray-Tracing Tree



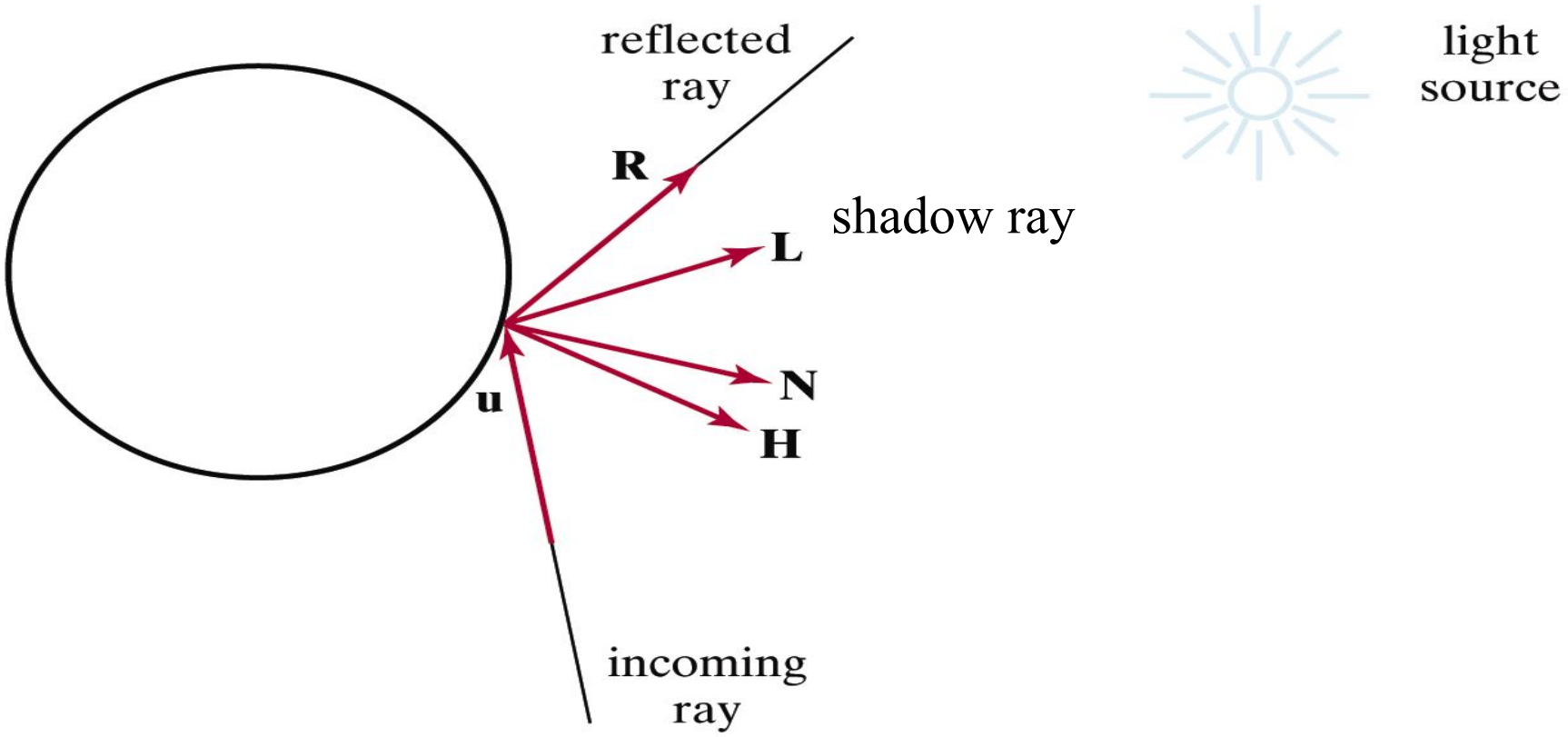
Ray Trace and Ray-Tracing tree

Basic Ray-Tracing Algorithm...

- Terminate a ray-tracing path on any one conditions:
  - Ray reach the preset maximum number of reflections
  - Ray strike a light source
  - Ray intersects no surfaces
- After the ray-tracing tree has been completed for a pixel, the intensity contributions to pixel are accumulated.
  - Start at terminal node (bottom) of tree and accumulate intensities at root node (nearest surface)
  - Surface intensity at each node is attenuated by the distance from the parent surface and added to the intensity of the parent surface.
  - Sum of the attenuated intensities at the root node is assigned to pixel.
  - Intensity of light source (If ray intersects non-reflecting light source)
  - Background intensity (If no surfaces are intersected, tree is empty)

- A surface intersected by a ray and the unit vectors needed for the reflected light-intensity calculations.
- Unit vector **u** is in the direction of the ray point, N is the unit surface normal, R is the unit reflection vector, **L** is the unit vector pointing to the light source, and H is the unit vector halfway between V and L.
- The path along L is referred to as the **shadow** ray.
- If any object is intersects the shadow ray between the surface and the point light source.

- For reflected and transmitted rays,
- At each surface intersection the illumination model is invoked to determine the surface intensity contribution



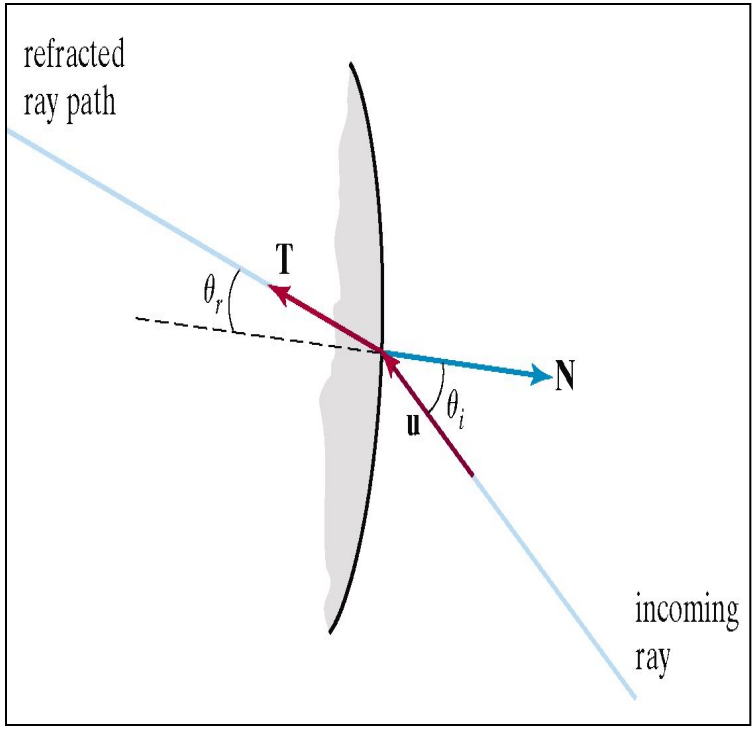
- The specular reflection direction for the secondary ray path depend on the surface normal and the incoming ray direction.

$$R = u - (2u \cdot N)N$$

- For the transparent surface the intensity contribution from light transmitted through the material is found

$$T = \frac{\eta_i}{\eta_r} u - \left(\cos\theta_r - \frac{\eta_i}{\eta_r} \cos\theta_i \right) N$$

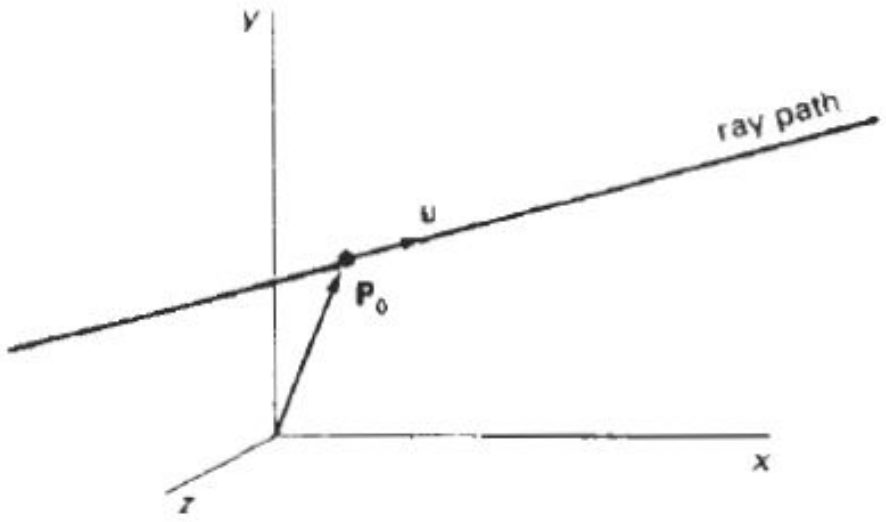
$$\cos\theta_r = \sqrt{1 - \left(\frac{\eta_i}{\eta_r} \right)^2 (1 - \cos^2\theta_i)}$$



Ray surface intersection calculations

- A ray can be described with an initial position **Po** and unit direction vector u.
- The coordinates of any point P along the ray at a distance**s** from **Po** is computed from the **ray equation:**

$$\mathbf{P} = \mathbf{P}_0 + s\mathbf{u}$$



- Initially, **Po** can be set to the position of the pixel on the projection plane, or it could be chosen to be the projection reference point.
- Unit vector **u** is initially obtained from the position of the pixel through which the ray passes and the projection reference point.

$$\mathbf{u} = \frac{\mathbf{P}_{pix} - \mathbf{P}_{prp}}{|\mathbf{P}_{pix} - \mathbf{P}_{prp}|}$$

- If we have a sphere of radius r and center position P_c, then any point P on the surface must satisfy the sphere equation:

$$|\mathbf{P} - \mathbf{P}_c|^2 - r^2 = 0$$

- Substituting the ray equation

$$|\mathbf{P}_0 - s\mathbf{u} - \mathbf{P}_c|^2 - r^2 = 0$$

- **Let P = P_c - P_0,** *and* expand the dot product, we obtain the quadratic equation

$$s^2 - 2(\mathbf{u} \cdot \Delta\mathbf{P})s + (|\Delta\mathbf{P}|^2 - r^2) = 0$$

whose solution is

$$s = \mathbf{u} \cdot \Delta\mathbf{P} \pm \sqrt{(\mathbf{u} \cdot \Delta\mathbf{P})^2 - |\Delta\mathbf{P}|^2 + r^2}$$

- If the **discriminant** is negative, the ray does not intersect the sphere. Otherwise, the surface intersection coordinates are obtained from the ray equation
- For small spheres that ***are*** far from the initial ray position

$$r^2 << |\Delta\mathbf{P}|^2$$

$$s = \mathbf{u} \cdot \Delta\mathbf{P} \pm \sqrt{r^2 - |\Delta\mathbf{P} - (\mathbf{u} \cdot \Delta\mathbf{P})\mathbf{u}|^2}$$

- Polyhedral **require** more processing than spheres to locate surface intersections.

$$\mathbf{u} \cdot \mathbf{N} < 0$$

- the plane equation N.P=-D

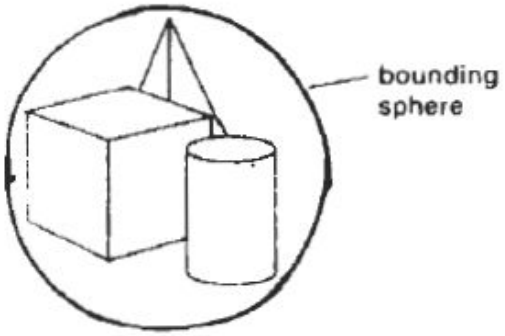
$$\mathbf{N} \cdot (\mathbf{P}_0 + s\mathbf{u}) = -D$$

- And the distance from the initial ray position to the plane is

$$s = -\frac{D + \mathbf{N} \cdot \mathbf{P}_0}{\mathbf{N} \cdot \mathbf{u}}$$

Reducing Object-Intersection Calculations

- One method for reducing the intersection calculations is to enclose groups of adjacent objects within a bounding volume, such as a sphere or a box
- We can then test for ray intersections with the bounding volume.



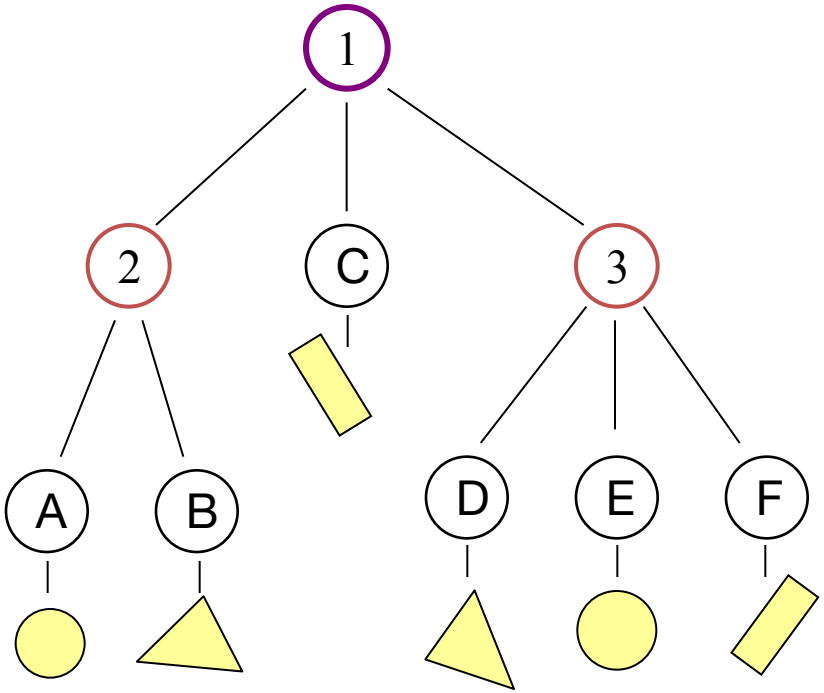
- If the ray does not intersect the bounding object, we can eliminate the intersection tests with the enclosed surfaces.

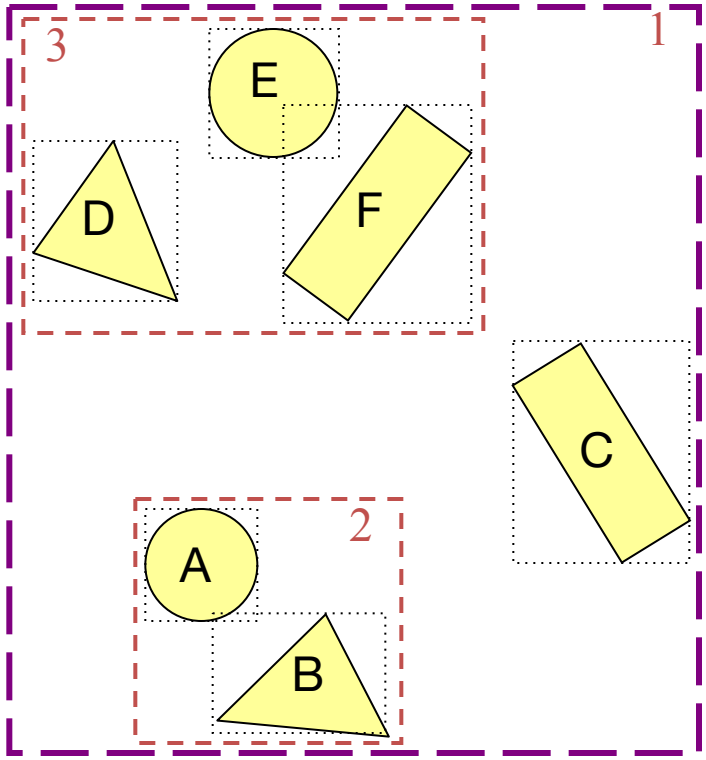
- We enclose several bounding volumes within a larger volume and carry out the intersection tests hierarchically.
- First, we test the outer bounding volume; then, if necessary, we test the smaller inner bounding volumes; and so on.

.

Bounding Volume Hierarchies

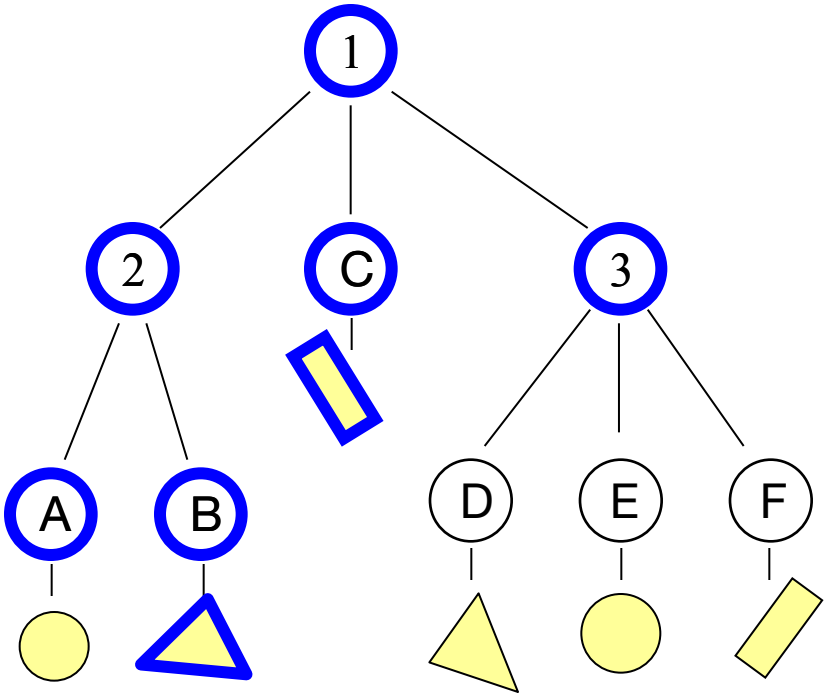
- Build hierarchy of bounding volumes
  - Bounding volume of interior node contains all children

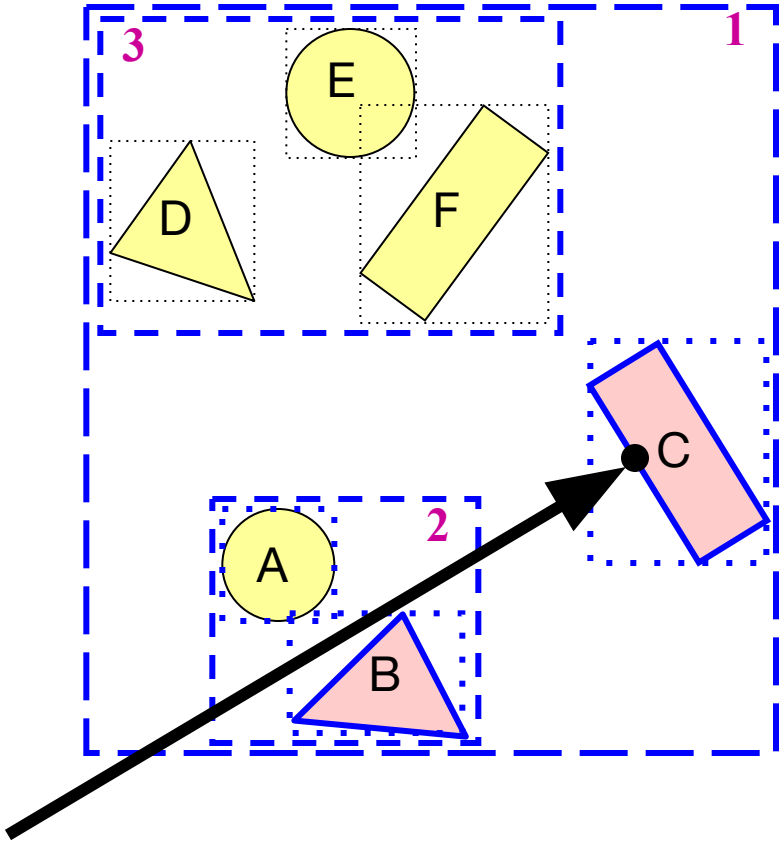




Bounding Volume Hierarchies

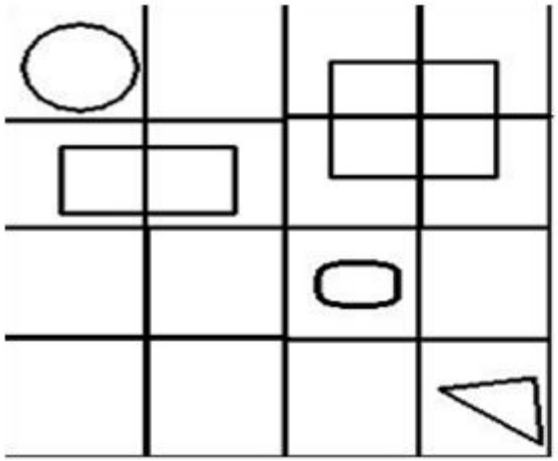
- Use hierarchy to accelerate ray intersections
  - Intersect node contents only if it hit bounding volume



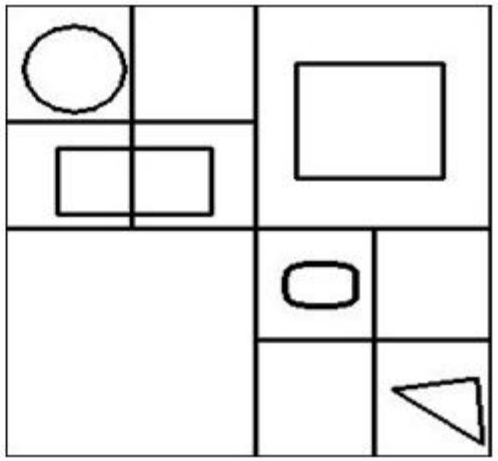


Space-Subdivision Methods

- We can enclose a scene within a cube, then we successively subdivide the cube until each sub region (cell) contains no more than a preset maximum number of surfaces.
- Space subdivision of the cube can be stored in an octree or in a binary-partition tree.
- In addition, we can perform a ***uniform subdivision*** by dividing the cube into eight equal-size octants at each step, or we can perform an ***adaptive subdivision*** and subdivide only those regions of the cube containing objects.



Uniform subdivision

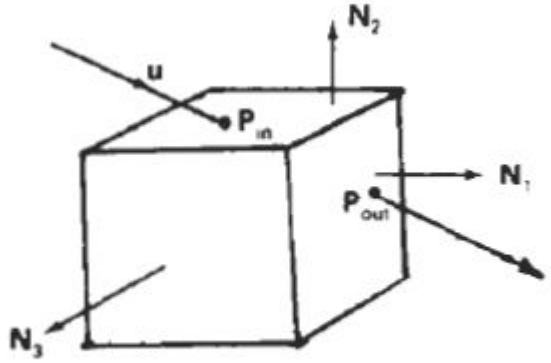


Adaptive subdivision

- Then trace rays through the individual cells of the cube, performing intersection tests only within those cells containing surfaces.
- The first object surface intersected by a ray is the visible surface for that ray.
- There is a trade-off between the cell size and the number of surfaces per cell.
- If we set the maximum number of surfaces per cell too low, cell size can become so small that much of the savings in reduced intersection tests goes into cell-traversal processing.

- Once we calculate the intersection point on the front face of the cube, we determine the initial cell intersection by checking the intersection coordinates against the cell boundary positions.
- We then need to process the ray through the cells by determining the entry and exit points for each cell traversed by the ray until we intersect an object surface or exit the cube enclosing the scene

- Given a ray direction **u** and a ray entry position **Pi,** for a cell, the potential exit faces are those for which



$$\mathbf{u} \cdot \mathbf{N}_k > 0$$

- If the normal vectors for the cell faces are aligned with the coordinates axes, then

$$N_k = \begin{cases} (\pm 1, 0, 0) \\ (0, \pm 1, 0) \\ (0, 0, \pm 1) \end{cases}$$

- we only need to check the sign of each component of u to determine the three candidate exit planes.
- The exit position on each candidate plane is obtained **Ray-Tracing Methods** from the ray equation:

$$\mathbf{P}_{out,k} = \mathbf{P}_{in} + s_k \mathbf{u}$$

- Where s_k is the distance along the ray from P_{in}, to $P_{out,k}$. Substituting the ray equation into the plane equation for each cell face:

$$\mathbf{N}_k \cdot \mathbf{P}_{out,k} = -D$$

- we can solve for the ray distance to each candidate exit face as

$$s_k = \frac{-D - \mathbf{N}_k \cdot \mathbf{P}_{in}}{\mathbf{N}_k \cdot \mathbf{u}}$$

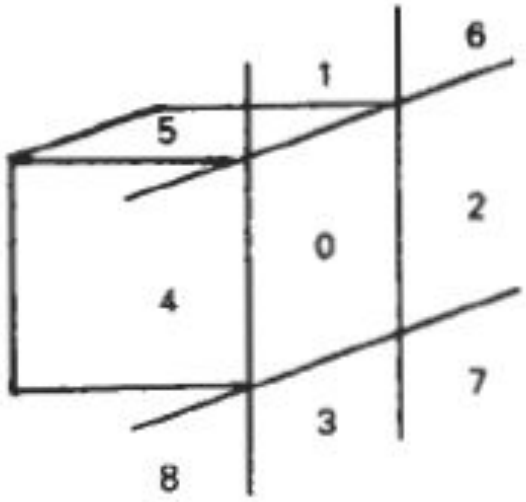
- and then select smallest s_k. This calculation can be simplified if the normal vectors **N,** are aligned with the coordinate axes.
- For example, if a candidate normal vector is (1, **0,** *0),* then for that plane we have

$$s_k = \frac{x_k - x_0}{u_x}$$

- where **u** = (ux , uy, ***uz),*** and ***xk*** is the value of the right **boundary** face for the cell.

- One possibility is to take a trial exit plane k as the one perpendicular to the direction of the largest component of u.
- The sector on the trial plane containing Pout,k determines the true exit plane.
- If the intersection point is in sector 0, the trial plane is the true exit plane and we are done.
- If the intersection point is sector 1, the true exit plane is the top plane
- Sector 3 identifies the bottom plane as the true exit plane; and sectors 4 and 2 identify the true exit plane as the left and right cell planes, respectively.

- When the trial exit point falls in sector **5,6,7,** or 8, we need to carry out two additional intersection calculations to identify the true exit plane.



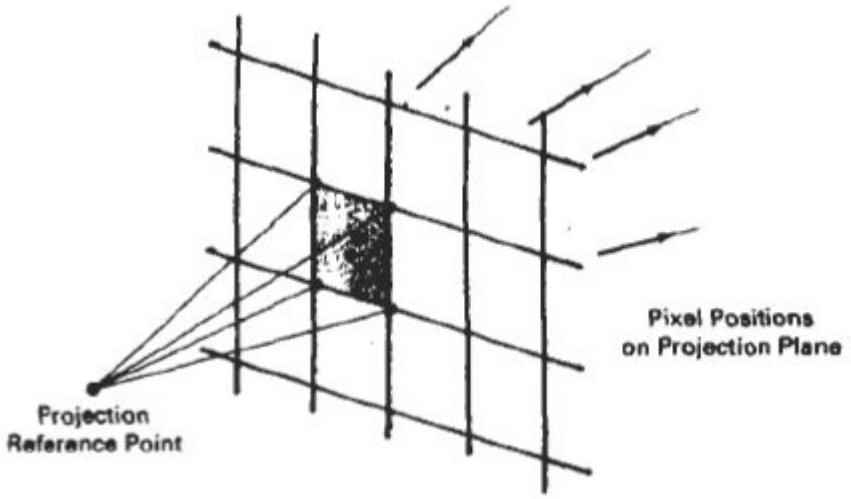
- Light-buffer technique, a form of spatial partitioning.
- Here, a **cube** is centered on each point light source, and each side of the cube is partitioned with a grid of squares, A sorted list of objects that are visible to the light through each square is then maintained by the ray tracer to speed up processing of shadow rays.
- To determine surface-illumination effects, the square for each shadow ray is computed and the shadow ray is then processed against the list of objects for that square.

Antialiased Ray Tracing

- Two **basic** techiniques for antialiasing in ray-tracing algorithms are ***supersampling*** and ***adaptive*** sampling
- In supersampling and adaptive sampling the pixel is treated as a finite square area instead of a single point.
- Supersampling uses multiple, evenly spaced rays (samples) over each pixel area. Adaptive sampling **uses** unevenly spaced rays in some regions of the pixel area.

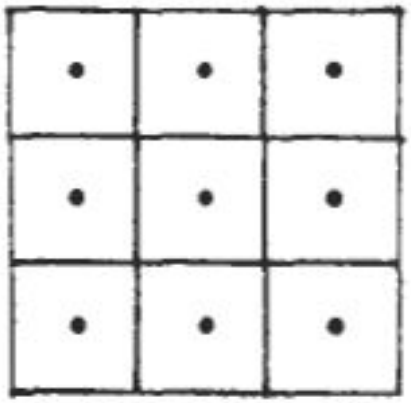
- Another method for sampling is to randomly distribute the rays over the pixel area.
- When multiple rays per pixel are used, the intensities of the pixel rays are averaged to produce the overall pixel intensity.

- One ray is generated through each coner of the pixel.
- If the intensities for the four rays are not approximately equal, or if some small object lies between the four rays, we
- divide the pixel area into subpixels and repeat the process.

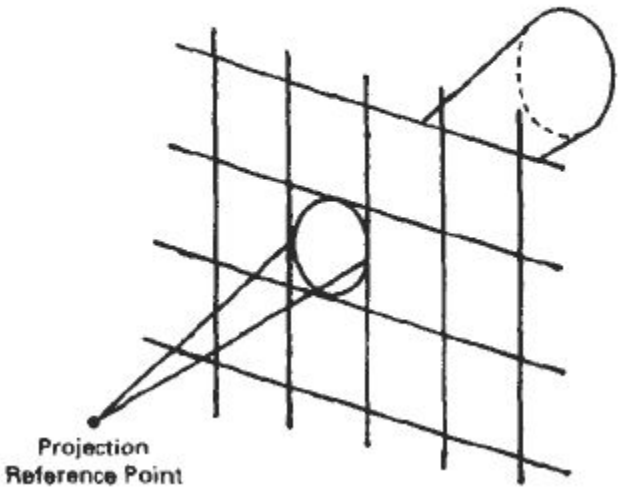


- The pixel in is divided into nine subpixels using **16** rays, one at each subpixel corner.
- Adaptive sampling is then used to further subdivide those subpixels that do not have nearly equal-intensity rays or that subtend some small object.
- **This** subdivision process can be continued until each subpixel has approximately equal-intensity rays or an upper bound, say, 256, has been reached for the number of rays per pixel.

- Instead of passing rays through pixel **corners,** we can generate rays through subpixel centres



- Another method for antialiasing displayed scenes is to treat a pixel ray as a cone,
- Only one ray is generated per pixel, but the ray now has a finite **cross** section.
- To determine the percent of pixel-area coverage with objects, we calculate the intersection of the pixel cone with the object surface.
- For a sphere, this quires finding the intersection of two circles. For a polyhedron, we must find the intersection of a circle with a polygon.

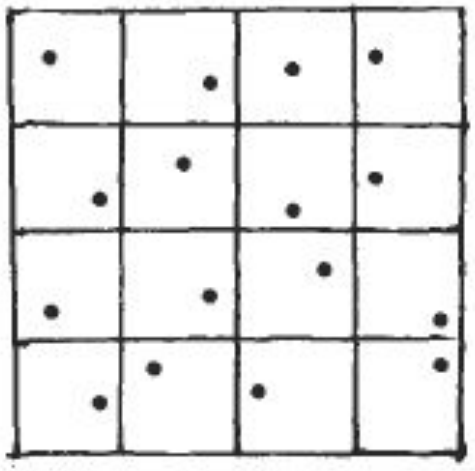


Fi
A

Distributed Ray Tracing

- **This** is a stochastic sampling method that randomly distributes rays according to the various parameters in an illumination model.
- Illumination parameters include pixel area, reflection and refraction directions, camera lens area, and time.
- Pixel sampling is accomplished by randomly distributing a number of rays over the pixel surface.
- Choosing ray positions completely at random, however, can result in the rays clustering together in a small -region of the pixel area, and leaving other parts of the pixel unsampled.

- **A** better approximation of the light distribution over a pixel area is obtained by using a technique called jittering on a regular subpixel grid.
- This is usually done by initially dividing the pixel area (a unit square) into the 16 subareas and generating a random jitter position in each subarea.

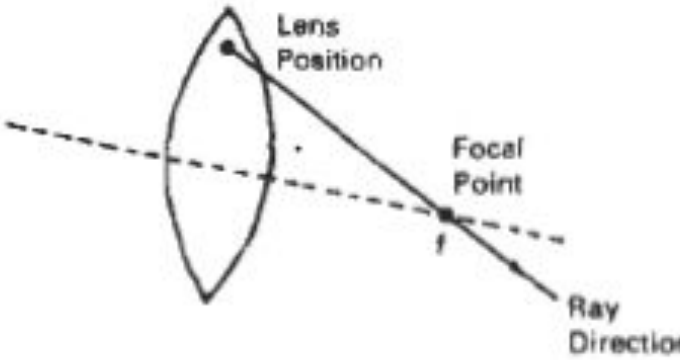


- The random ray positions are obtained by jittering the center coordinates of each subarea by small amounts, δ_x, and δ_y, where both δ_x, and δ_y are assigned values in the interval (-0.5,0.5). We then choose the ray position in a cell with center coordinates **(x, y)** as the jitter position ***(x*** + δ_x**,** y + δ_y **).**
- Integer codes 1 through 16 are randomly assigned to each of the 16 rays, and a table lookup is used to obtain values for the other parameters

- Each subpixel ray is then processed through the scene to determine the intensity contribution for that ray.
- The 16 ray intensities are then averaged to produce the overall pixel intensity. If the subpixel intensities vary too much, the pixel is further subdivided.
- To model camera-lens effects, we set a lens of assigned focal length f in front of the projection plane ,and distribute the subpixel rays over the lens area.
- Assuming we have 16 rays per pixel, we can subdivide the lens area into 16 zones.

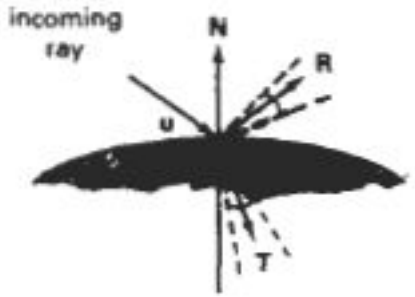
- Each ray is then sent to the zone corresponding to its assigned code.
- The ray position within the zone is set to a jittered position from the zone center.
- Then the ray is projected into the scene from the jittered zone position through the focal point of the lens.

- We locate the focal point for a ray at a distance f from the lens along the line from the center of the subpixel through the lens center



- Objects near the focal plane are projected as sharp images.
- Objects in front or in back of the focal plane are blurred. To obtain better displays of out-of focus objects, we increase the number of subpixel rays.

- Ray reflections at surface intersection points are distributed about the specular reflection direction R according to the assigned ray codes



- The maximum spread about R is divided into 16 angular zones, and each ray is reflected in a jittered position from the zone center corresponding to its integer code.

- We can use the Phong model, $\cos \varphi$, to determine the maximum reflection spread.
- If the material is transparent, refracted rays are distributed about the transmission direction T in a similar manner.